User Manual

Data Transformation Package

This manual provides an overview of the Data Transformation Package for managing data extraction, transformation, and loading.

Version: 5.3.SP36

Wolters Kluwer
CCH® Tagetik
wolterskluwer.com

Table of contents

Data Transformation Package (DTP) 1

Data Transformation Package (DTP) 1
Running the DTP 2
DTP reports 4
Parametric DTPs 5
Details of SQL filters in DTP operations 7
Data transformation package tab 8
New DTP window 9
DTP settings window 9
Clone 9
Clone DTP Window 9
Checking rules 10
Calculation rules 10
AW Parameters page 12
Special methods page 13
Checking rules page 14
Values page 15
Calculation rules page 15
DTP activity cockpit 16
Data processing is complete window 17

DTP activities 19

(DTP) activity definition 20
DTP Debug Activity 21
Duplicate Activity window 23
Activity Debug Window 25
Temporary Debug Partitions 26
Temporary Debug Partitions page 26
Definition of activity and operation loops 27
Uploads 28
Loadings Page 28
Update data model 29
Update rules 31

Write on another dataset 33
Export to Financial Workspace 33
Operations 34
Operations 35
Matching fields page 40
Advanced settings page 43
Ask AI for DTP 43
Insert from other dataset 46
Enrichment from another dataset 47
Enrichment with formula 49
Enrichment with formula field 49
Enrichment with Reference dataset 50
Enrichment with ranking 53
Insert with reference dataset 54
Insert with pivot transposition 54
Insert with unpivot transposition 56
Join from another dataset 57
Analytical allocations 58
Allocation details page 60
Special method 61
Check with formula 62
Check with dictionary 62
Check with rule 64
Check missing keys 64
Existence check 66
Check multiple keys 67
Loading to Analytical Workspace 68
Delete with formula 69
Delete multiple keys 69
Delete missing keys 70
Insert with calculation rule 72

XML Generator 77

XML Generator: Basic Concepts 77
Regulatory reporting page 77

Tag configuration page **78**

Data Transformation Package (DTP)

Contents

This section covers the following topics:

Data Transformation Package (DTP) 1
Running the DTP 2
DTP reports 4
Parametric DTPs 5
Details of SQL filters in DTP operations 7
Data transformation package tab 8
New DTP window 9
DTP settings window 9
Clone 9
Clone DTP Window 9
Checking rules 10
Calculation rules 10
AW Parameters page 12
Special methods page 13
Checking rules page 14
Values page 15
Calculation rules page 15
DTP activity cockpit 16
Data processing is complete window 17

Data Transformation Package (DTP)

Introduction

The Data Transformation Package is a calculation engine that runs data transformation processing in the context of Business Workflow tasks that can be used in processes. In particular, every DTP is made up of one or several *activities* comprising one or several *operations*.

To define a DTP, it is therefore necessary to create the relative activities, divided into operations. The DTP performs calculations on dedicated physical tables identified by typologies with the logical structure defined by the dataset.

Activity details

Each DTP can contain several tasks that are executed in the order defined by the user. The following elements must be defined for every activity:

- the target dataset, or the dataset in which the operations insert, edit or check data,
- a target dataset typology
- a priority level, or the order in which the operations will be run on the reference dataset (for example, a level 3 activity is run after the level 2 activity and before the level 4 activity).

Operation types

Operation type	Description
Insertion	Inserts data from a source dataset into a target dataset.
Enrichment	Enriches a target dataset with data from a source dataset.
Check	Runs checks on the target dataset data according to criteria defined by the user
Deletion	Deletes data from the target dataset according to user-defined criteria

Based on the operation type, it is possible to define calculation criteria for specific data. The operations are run in the order in which they are listed in the DTP. For details of the available operations, see Operations.

Parametric DTPs

When defining a DTP, it is possible to use parametric elements that are resolved at runtime.

For more details, see Parametric DTPs.

Restrictions on the duplication of activities

To create activities with similar properties, it is possible to duplicate activities present in the current DTP. Below are some restrictions and behaviours:

- If there are errors in the duplication of the activity or one of the operations that it contains, the entire data processing will be cancelled.
- Operations which contain *Matching fields* cannot bring those relationships with them if the target dataset is changed in the duplication operation. In such cases, the system searches the new dataset for any columns with the same name as the original target dataset. It then only matches columns with the same name as those saved in the matching fields.
- With *Loading to Analytical Workspace* operations, the operation is only copied if the target dataset matches the source dataset.
- With *Special Method* operations, the values are not copied in the operation.
- For *Allocation* operations, the details are not copied in the operation.
- Fields containing filters or formulas such as *Target filter*, *SQL formula*, or *Run condition* are copied even if they contain references to columns of the target dataset.
- The following duplications are not supported:
  - *Enrichment with formula field* operation
  - fields matched for the *Enrichment with reference table* operation
- Field relationships related to operations will be duplicated if the target dataset matches the source dataset or if the target dataset contains a fields with the same code as a field of the source dataset.
- Relationship fields with special **N** values, which automatically relate the remaining fields, and **F** values, which apply the formula to the remaining fields, are copied in any case.
- In case of upload > *Write to another dataset* or **Export Reporting layer to file** in the activity, the upload will only be copied if the target dataset/type pair matches the source dataset/type pair.
- In case of upload > **Export to Financial Workspace** in the activity, the upload will be copied even if no Financial Workspace mapping is configured for the target dataset.

Running the DTP

Run priorities

The process for running DTPs in various execution contexts is provided below.

Element	Description
Runs several DTPs simultaneously	By default, it is not possible to run, in the same execution context (Process, Scenario, Period, Entity), at the same time DTP data processes containing at least one identical dataset/type. Any data processing started during the data processing that is already in progress will be carried out once it has ended, in the run order. ***Note**: When data processing is run from a workflow task in parallel with a data processing already in progress, containing at least one identical dataset/type, the system displays a window in which the user can choose whether to cancel the launch, retry or put the data processing in the queue. In all cases (workflow, form, API, job) the system performs a check on the resources used by the **DTP** and **Update Financial Workspace from Analytical Workspace** running data processes; if there are resources already used by the current data processes, the booking is done automatically. The waiting time will be indicated in the audit under 'Data processing started' with the heading 'Wait before the run'.*
Run DTPs with the *run independently from context* option active	A DTP containing tasks with the *run independently from context* option will never be queued, but run in parallel with any other data processing that is in progress. DTPs with the *run independently from context* option can be run in parallel with other data processing, whether or not they are associated with a task with "run independently from context".

Element	Description
Run DTPs simultaneously from task groups or workflows in grid	By default, concurrent data processing of the same type is not supported, whether the data processing is run from task groups or workflows in grid. Data processing by task group runs the different DTP tasks of the group in series, regardless of the order of the workflow. Launch workflow in grid: enables the simultaneous launch of different DTPs on different list elements
Run DTPs simultaneously from task groups or workflows in grid with tasks with the *run independently from context* option active and tasks on which the option is not active.	When the *run independently from context* option is only active on some workflow tasks, and several DTPs are run simultaneously, the system performs concurrency checks. Based on the outcome of the checks, run from tasks that do not have the option active, the data processing is run or added to the queue.
Run DTPs with activities or operations containing SQL scripts	Opens a window to define the parameters required at runtime. For further details on the definition of those parameters, see in the Reporting module.

In all execution contexts, if the DTP contains personal data, the DTP data processing can only be run correctly if the data are properly encrypted (see Personal Data Protection).

Run the DTP for entity nodes

The DTP data processing involves data divided by Scenario/Period and Entity. When a data process is run for entity nodes, the DTP may behave differently based on the setup:

If...	Then...
all the DTP targets are partitioned by Business Cycle and Entity.	the DTP is processed individually for each entity. The data processing can be run in parallel if all the execution conditions are satisfied. To run multiple data processing, all the entities must have the same workflow as the selected node.
the datasets are not partitioned by Business Cycle and Entity	the DTP is run just once, with a data process containing the data of all the entities present in the node.

When the system detects a case that is different from those listed, it blocks the data process and reports the error.

> ***Note**: when several entities are selected in the contributor cockpit, the DTP execution can only be completed if the tasks selected in the detailed task list do not run the same DTP for the same entity.*

To run the DTP data processing by entity node from forms, the following syntax is necessary: *<$Entity.Dimension.hierarchyCode>|<$Entity(false).code>*.

Run the DTP independently from the context

On DTP tasks it is possible, via forms, to activate the *Run independently from context* option, which allows you to run them in parallel with any other DTP data processing.

With the option active, the data on partitions with typologies other than *Reporting layer* will not be saved.

This option prevents the use of partitions with *Calculation layer* typology in read/write mode if these are the result of data processing originating from other DTPs.

Run the DTP from Jobs

When the execution of DTP data processing is run from a job, the statuses on the DTP cockpit are not updated. Both with jobs g run from the processes cockpit and with jobs run from a job's run menu, the system saves neither the anomalies deriving from the checking operations nor the operation start and end dates. The information on the last execution prior to being run from a job remains visible.

Data processing ended with anomalies

When the execution of DTP data processing involves check operations, these generate anomalies if the check fails.

Editability checks

When DTP is run, it does not apply editability checks on data.

Thus, for example, it is possible to process submitted entities, or data locked at process level (e.g. with the 'Lock Data' function).

Editability checks are applied instead, for example:

- when loading data via Datasource Mapping;
- during the Update Financial Workspace phase;
- when writing to the Reporting Layer, if the Loading operation 'Write to another dataset' is used.

DTP reports

From the contributor cockpit, the following reports can be launched from the action panel by clicking on the DTP item, but only when executed from lowest level element and if the execution is standard (i.e., context independent).

Note:

- *these reports are not accessible from the Detail summary tab.*
- *In the audit, you can review anomalies, but only the first 10 are displayed.*

Checking rules report

The DTP's *checking rules* report can be run from the contributor cockpit. In the cockpit, details of the dataset/typology paid on which the checking rules have been run are shown at the end of the activity. A list of the checking rules run by the DTP can be exported for every record, in XLSX format. The generated file name will have the following structure:

REGOLEDICONTROLLO_<COD_SCENARIO>_<COD_PERIODO>_<COD_AZIENDA>_<COD_DATASET>_<COD_TIPOLOGIA>_<TIMESTAMP>.XLSX

Warnings report

From the contributor cockpit, it is possible to run the *anomaly report* from the *Data Transformation Package* task by clicking on the DTP button in the action bar.

This report shows errors found in DTP operations. Specifically, if a task ends with anomalies, it displays the list of operations that caused those anomalies during the last run, along with the number of anomalous records found.

When an operation with anomalies has performed a *check* on a dataset, it is possible to download an XLSX file containing details of the dataset rows with anomalies.

The table columns show the following information:

- the DTP that generated the error
- the activity
- the Target dataset
- the target Type
- the n-tuple of LOOP values belonging to the activity iteration
- information on the checking operation that found the error
- the operation type
- the n-tuple of LOOP values belonging to the operation iteration

- the user that ran the data processing
- the data processing run time

The XLSX file shows all the errors found since all the DTPs present in the task workflow were last run.

In the rows above the table with the anomalous values of the operation, the following information is displayed:

- the first row of the file contains the code and description of the operation
- if the operation is part of an activity that has an associated LOOP, the n-tuple of the activity's LOOP iteration values is shown, followed by the n-tuple of the operation's LOOP iteration values if it has an associated LOOP.
- the error message defined on the operation page.

The table with all anomalous data is shown below.

Parametric DTPs

Introduction

When defining a DTP, it is possible to use parametric elements that are resolved at runtime.

Specifically:

- Attributes, which are resolved by the system automatically, based on the specifications entered in the DTP definition step.
- Parameters defined for reporting or data entry purposes, which must be defined at runtime by the user running the data processing.
- Parameters defined in the DTP, which are defined by the administrator in the DTP definition step and resolved automatically at runtime.

Attributes

In DTP fields that accept SQL formulas (e.g. filters and formulas for defining dataset fields), it is possible to refer to attributes related to CCH Tagetik lists (e.g. Scenarios).

For example, attributes can be used to filter a field based on the currency of the entity on which the DTP is being run or define a field with the description from the Previous Scenario before the run scenario.

Attributes must be defined with the syntax used in the reporting for the cell fields and applicable to all dimensional attributes.

The syntax must be between braces :${}.

Example

To filter the reading of data from a dataset, only for records whose CURRENCY field matches the currency of the entity, which you have selected in the process cockpit when running the DTP, you must:

1. Define an *Insert from another dataset* operation.
2. Specify the *CURRENCY = ${$Entity.currency.code}* clause in the *Filter* field.

Example

To define the PREVIOUS_SCENARIO field with the code of the previous scenario that came before the run scenario in an *Enrichment with formula* operation, you must specify the *${$Scenario.prev.code}* syntax in the *SQL formula* field.

Attributes can also be used to define query type database objects in the datasource mapping, special methods and dictionaries.

Parameters for reporting

In DTP fields that support SQL formulas, it is possible to refer to reporting parameters (see in the Reporting module).

***Note**: it is possible to refer to all types of parameters (dimensional, dictionary, text, numeric, date or flag).*

If a DTP uses reporting parameters, when it is run the system will ask you to select one of the values defined as the parameter's options. It is also possible to define a default value.

Dependencies between parameters can be defined in the DTP using advanced selection scripts. Only explicit dependencies are allowed.

Dependent parameters defined in the parameter definition window, Excel add-in side, can also be used within the DTP. Thus, when running a DTP with dependent parameters, the quick search combo box will only show the values that are consistent with the selected dimension.

Example

It is possible to make one or several DTP operations optional. To do this, a Flag type reporting parameter must be defined. Then, in the DTP, in the activity's *Run condition* field, define an activity that encompasses all optional Operations using the following syntax: *${@Parameter_code}*.

At runtime, the user selects a flag to decide whether or not to define the parameter.

Reporting parameters return codes reported in inverted commas, with the exception of the syntax *DATASETPARTITION()*, which refers to the name of a partition. This is generally used in the *FROM* clause of a *query* to refer to the physical name of a partition.

***Note**: If the syntax DATASETPARTITION() is used in the FROM clause in a query and the body of the query contains parameters, and the parameters are not correctly defined when running, the system will show an incorrect query syntax warning. In this case, it is advisable to carry out checks on the correct use of the parameters.*

IMPORTANT: It is not possible to use reporting parameters in the default AW parameter formula and in the Analytical workspace dictionaries. If the parameter is used in dictionary queries, it will not be displayed at runtime. If the parameter is used in an object that supports it, it can be defined on the dictionary.

Parameters defined in the DTP

Parameters defined in the DTP can identify a value or an SQL formula, which will be resolved at runtime and can be used in any DTP field that accepts SQL formulas.

For example, they can be defined to use the same DTP on different processes, basing the data processing logic on the current process. Or they can be defined to determine the filters to be applied to the data processing, or to define a certain dataset field.

Element	Description
Workspace parameters	Parameters defined at the workspace level and referred to using the syntax *:@NOME_PARAMETRO@*. Its value, or the SQL formula, is defined on creation. When the DTP is run, the parameter is replaced with the value that it takes on or. based on the parametrization, its value is requested at launch.
System parameters	See "System parameters" below.
Special method parameters	Parameters defined in the special method and referred to in the SQL formula using the syntax *&NOME_PARAMETRO&*. The value or SQL formula with which to replace the parameters is defined on the operation, in the special method settings (see Special method).

***IMPORTANT**: in SQL formulas, both reporting parameters and DTP parameters can be used simultaneously. No reporting parameters can be used when defining the default formula of the AW parameters or the value to be assigned to the parameter.*

***IMPORTANT**: When the parameter refers to a dimension not belonging to the launch context, the parameter is not automatically resolved.*

For example, if the user launches a DTP or QDL with a parameter of #ENTITIES# type and the process launch phase is by analytical dimension, the parameter will not be resolved and will be left untouched.

***Note**: In the case of execution on the process cockpit with dimension by step, the same limitations apply as for the Loading to Analytical Workspace task.*

System parameters

Predefined parameters automatically resolved when the DTP is run depending on the execution context. They can be referred to using the syntax *#NOME_PARAMETRO#*. There are the following system parameters:

- *#USER#*: user who run the data processing
- *#AW_CODE#*: workspace code of the DTP run in the data processing
- *#PROCESS#*: process code on which the data processing was run
- *#ENTITIES#*: list of entities for which the request to run the DTP was made (separated by commas)
- *#ANALYTICAL_ DIMENSIONS#*: list of *analytical dimensions* on which DTP execution was requested, separated by commas;
- *#CUSTOM_DIMENSIONS#*: list of *custom dimensions* on which DTP execution was requested, separated by commas;
- *#PERIODS#*: list of periods for which the request to run the DTP was made
- *#SCENARIOS#*: list of scenarios for which the request to run the DTP was made
- *#SCENARIOPERIODS#*: scenario/period on which execution was requested (with single-submission processes, it contains the list of scenarios/periods sorted by increasing reference date)
- #REFERENCE_DATE#: *reference date of the scenario/period* of the execution or of the business cycle in case of a single submission process
- *#TARGET_PARTITION#*: target dataset partition of the activity being run
- *#ITERATION_VALUE#*: iteration value within an activity loop
- *#ITERATION_VALUE_OPERATIONS#*: iteration value within an operation loop
- *#AIV#:* returns the n-tuple of iteration values within an activity loop.
  It may refer to a field (*#AIV.FIELD#*): the value of the FIELD field of the corresponding iteration within an activity loop is returned.
  It may refer to the alias (*#AIV.ALIAS#*)
- *#OIV#:* returns the n-tuple of iteration values within a loop of the operation.
  It may refer to a field (*#OIV.FIELD#*): the value of the FIELD field of the corresponding iteration within a loop of the operation is returned.
  You can refer to the alias (*#OIV.ALIAS#*)
- *#PROCESS_STEP#*: code of the step to which the running DTP belongs.

***Note**: The codes returned by system parameters are not reported in inverted commas. For the codes to be reported in inverted commas, you must specify that condition in the SQL formula that calls the parameter.*

Details of SQL filters in DTP operations

Dimensional attributed in SQL filters

It is possible to refer to CCH Tagetik dimensions in the execution conditions and SQL filters in DTP activities and operations. The references must be defined with the syntax of the cell fields used in the reporting.

Example: the syntax for checking for the presence of entity EN_00, as a condition for the execution of an activity, is as follows: ${$Entity.code} = 'EN_00'. If the outcome is positive, the result is 'EN_00' = 'EN_00' and the activity is run.

The syntax for checking, as an execution condition, for the presence of an entity EN_00 in a node, is 'EN_00' in (${$Entity.code}).

The system checks that entity EN_00 is among those present in the activity run node.

The cell field is replaced with the value resulting from the checking of the condition reported in inverted commas. If the outcome of the check is a set of values, the cell field is replaced by the set of values in inverted commas separated by commas.

The syntax *${@PARAMETRO}* can be used, in the cell field, to refer to the parameters present in the form parameters definition (see in the Reporting module).

***Note**: the outcome of the DATASETPARTITION cell field check is not reported in quotes as it is a table name.*

Syntax supported in SQL fields

In fields that support SQL language the DTP allows the following syntax, converted in the run step.

Element	Description
System parameters	See "System parameters" on page 6.
@PARAMETRO@	Parameters defined at the workspace level, which take on the value indicated in the default formula present in the definition of the same parameter.
${@PARAMETRO}	Global parameters defined in forms, supported by SQL scripts defined at the Activity, Operation, Matching fields and Special methods level.
Unified SQL syntax	See Unified SQL syntax.

Data transformation package tab

Application database > Analytical Workspaces tile > workspace tile > Data transformation package tab

Application database > Navbar > Analytic Information hub > workspace tile > Data transformation package tab

Application Database > Navigation Panel > Analytic Information Hub > workspace tile > Data transformation package tab

Purpose of the tab

This tab allows you to manage workspace DTPs.

Tab structure

Element	Description
DTP tiles	These allow you to access the content of the relative DTP. By default they are sorted by description.
DTP management buttons	These allow you to manage DTPs.
Layout management buttons	These allow you to customise the page layout. ***Note**: by default, it displays up to 50 tiles sorted by description.* See Layout management buttons.

DTP management buttons

Element	Description
⊕ **New DTP**	Opens a window to add a new DTP. See DTP Settings window.
AW parameters	Opens a page to manage the parameters of Analytical Workspaces. See AW parameters page.
Special methods	Opens a page to manage special methods. See Special methods page.
Checking rules	Opens a page to manage checking rules. See Checking rules page.
Calculation rules	Opens a page to manage calculation rules. See Calculation Rules page.

Buttons on the DTP tile

The following buttons are accessible from the ⋮ menu in the right-hand corner of the tile.

Element	Description
Settings	Opens the DTP properties window. See DTP Settings window.
Clone	Opens the window for cloning the DTP. See Clone DTP.

New DTP window

Application database > Analytical Workspaces tile > workspace tile > Data transformation package tab > ⊕ New DTP

Purpose of window

This window allows you to create a Data Transformation Package.

DTP settings window

Application database > Analytical Workspaces tile > workspace tile > Data Transformation Package tab > (DTP tile) ⋮ > Settings

Purpose of window

This window allows you to manage properties or delete the DTP.

Buttons in the window

Element	Description
Delete	Deletes the DTP. ***IMPORTANT**: once a DTP has been deleted, it will no longer be possible to recover any of its data or settings.*

Clone

Introduction

The **Clone DTP** function allows a DTP to be copied from an existing one by duplicating its content and parameterisation.
The generated DTP is visible to users for whom the source workspace is visible.

Clone DTP Window

Application Database > Navigation Panel > Analytic Information Hub > workspace tile > Data transformation package tab > (DTP tile) ⋮ > Clone

Purpose of window

This window allows you to clone a DTP from an existing one, duplicating its contents.

Fields in window

Element	Description
Code	Code of the new DTP.
Description	Description of the new DTP. If several supported languages are enabled, at least one description is mandatory.

Buttons in the window

Element	Description
Cancel	To cancel the clone
Run	Enabled when a code and a description are inserted. Runs the data processing. At the end of the data processing, the system shows the created DTP and an audit window with the results of the data processing. In the event of an error, the entire data processing is cancelled and the DTP is not cloned.

Checking rules

Introduction

Checking rules allow you to validate data present in a dataset according to one or several criteria defined by the user and based on the dataset fields.

The following check elements can be defined for each rule:

- the type of check to be performed, the operator and the elements to be checked, potentially filtered
- a set of condition fields that contains dataset fields on which filters can be set in order to identify the values to be checked
- a set of grouping fields which contains the combination of dimensions with which to group the values to be checked

Types of checking rule

Checking rules are divided into the following types.

Rule type	Description
Existence rule	Checks that the value of every set of records is present and not 0.
Coherence rule	Checks that the value for the second set of data is also present, in addition to checking that the value of the first set of data is present and not 0.
Congruence rule	Checks, through comparison and according to the defined operator, that the difference between the values obtained from the first and second set of data is within the specified threshold.

Calculation rules

Introduction

Calculation rules allow you to run complex calculation operations and can be written with syntax similar to MD syntax.

***Note**: operation only available from version SP25, which can only be viewed and saved with the ESG licence.*

The function is only available if function "0004167 - Calculation rules" is enabled for the user.

Each calculation rule is based on the definition of operands, with their own logical dimensions and script dimensions that use similar syntax to MD syntax. Sets of calculation rules can be associated with an "Insert with calculation rule" DTP operation.

Script syntax

The syntax used to express each script is based on the Custom MD calculations syntax. With the following conventions:

- The script always starts with the notation: **TGKML 1.0** and is made up of several statements.
- In a script, statements are separated by ";".
- In every statement, for the assignment use: ":=".
- The statement must have one assignment only and must finish in ";".

- 2 cubes may be used: [TARGET] and [SOURCE], which indicate the write dataset and the read dataset, respectively.
- The cube is always mandatory for every operand and must be indicated as [cube].[operand].
- [TARGET] must only be present to the left of the assignment ([TARGET].[operand] and represents the output operand).
- Only an output operand can be present to the left of the assignment.
- [SOURCE] must only be present to the right of the assignment ([SOURCE].[operand] and represents an input operand).
- To insert a comment, use "//": any characters following the // symbol until the end of the row will be ignored. Comments can be inserted at any point in the script.
- An assignment is always of the following type: [TARGET].[operand] := <expression>, where the expression may be a simple assignment like: [SOURCE].[operand], constant, or an expression that includes mathematical operations or language-specific operations (fixed point, bound, filter, etc.).
- Arithmetic operations can be used: +, -, *, / and the expression may contain mathematical operations with scalars.
- Parentheses ( ) can be used to indicate the expression's execution order.

Logical dimensions window

This window contains a list of the logical dimensions associated with the operands. By default, the list is sorted by the script operand code column. Every operand, whether input or output, can be associated with several logical dimensions (e.g. GENDER, GEOGRAPHICAL_AREA, etc.) and each logical dimension must be mapped to a dataset field.

For every statement, the calculation identifies the information of the logical dimensions defined for the operands of which it is made up (input/output).

If...	Then...
the input operands if which it is made up have the same logical dimensions.	the operands are **uniform**.
at least one of the input operands' logical dimensions is different.	the operands are **non-uniform** or **irregular**.
no logical dimensions are shared.	the operands are **separate**.
an operand does not have rows defined in the logical dimensions table.	the operand has **no logical dimensions**.

Functionalities supported by the calculation rules

A list of the functions that can be run through the calculation rules is provided below.

Element	Description
Copy	Simple copy of the input operand onto the output operand (both with uniform logical dimensions and non-uniform logical dimensions).
Arithmetic expressions (addition, subtraction, product, division)	between input operands (with homogeneous, inhomogeneous and disjointed logical dimensions). **If the outcome of the formula is 0, the row is produced in output in any case.**
Override on output operand	Override on the output operand for one or several logical dimensions.
Read from Fixed Point	Read from fixed point: • Of a value of the logical dimension • On several different logical dimensions separated by commas • .All • .NotNull • .Null

Element	Description
Filter on logical dimensions of input operands	Filter on the logical dimensions of the input operands of a value or list of values separated by commas.
Bound on input operands.	
Temporary calculations '@'	Saving temporary calculation using the notation [TARGET].@MyTemp := ...
DiscardZeroes	
NotNode	
RoundNode	
TotalAmount	
UnaryMinusNode	
Max_FunctionNode	
Min_FunctionNode	
Sum_FunctionNode	

Additional features

Element	Description
Abs (absolute value function)	gives the result without sign of the expression passed as input. The syntax to be used is ... := **Abs (espr)**
If then else	allows the conditional construct to be used to evaluate an expression and drive the output: it is up to the user to manage the homogeneity of the dimensions used. The syntax to be used is ... := **if (*cond*) then *espr1* [else *espr2*] end;**
Median	expresses the median of the set of values. The syntax to be used is ... := **Median(<expression>, logical dimension1, ..., logical dimension N);**
Average (average function)	arithmetic mean of the values. The syntax to be used is ... := **Average(<expression>, logical dimension1, ..., logical dimension N);**

AW Parameters page

Application database > Analytical Workspaces tile > workspace tile > Data transformation package tab > AW parameters

Purpose of the page

This page allows the administrator to define parameters in the DTP when defining the DTP itself, using a value or SQL formula which will be resolved automatically at *runtime*. These values can be used in any field of the DTP that accepts SQL clauses.

They are defined at the Workspace level and referred to using the notation *@NOME_PARAMETRO@*.

Page fields

Element	Description
Parameter	Parameter code. Only combinations of uppercase letters, numbers and "_" are permitted.
Description	Parameter description
Default Formula	Default formula used to define the parameter
Ask at runtime	When active, the parameter must be defined when the DTP is run. When inactive, the parameter is defined with the value of the default formula.

Buttons in tab

Element	Description
	Add a parameter.
	Duplicates the selected parameter.
	Deletes the selected parameter.

Special methods page

Application database > Analytical Workspaces tile > workspace tile > Data transformation package tab > Special methods

Purpose of the page

This page allows you to define special methods, or specific method parameters and the syntax to be run in the DTP operation.

For more information on special methods, see Special Method.

Tabs on the page

Element	Description
Attributes	Allows you to manage the properties of the special method and define their parameters associated with the operation.
SQL Server	Allows you to insert the special method syntax. The syntax must be entered in the tab corresponding to the server in use. For more details, see Parameters supported in the operation syntax.
Oracle	
PostgreSql	
Hana	

Attributes tab

Element	Description
Special method	Special method code
Description	Description of the special method
Disable concurrency checks	When active, the method can be run in parallel with other activities. By default, special methods are run in isolation.
Parameters	(Only visible after the **Special method** and **Description** fields have been saved. Allows you to define specific parameters for the special method, which can be customised in the operation.

Parameters Pane

Element	Description
Code	Parameter code.
Description	Parameter description.
Type	• **Dataset type**: defines a parameter by which the user can select the dataset and type on which to run the operation. • **String**: defines a parameter in which the user can insert a string of text.

Buttons in tab

Element	Description
	Adds a special method.
	Duplicates the selected special method.
	Deletes the selected special method.

Checking rules page

Application database > Analytical Workspaces tile > workspace tile > Data transformation package tab > Checking rules

Purpose of the page

This page allows you to define checking rules to validate data present in a Dataset according to certain criteria.

Attributes tab

Element	Description
Checking rule	Checking rule code, unique for the Analytical Workspace
Description	Description of the checking rule
Enable	Enables use of the rule when running the DTP.
Type	Allows you to select the type of rule so it can only be run when relevant to the check type. The possible values are: • **Existence** • **Coherence** • **Congruence**
Operator	Allows you to select the values check operator. The available options will depend on the **Type** value. **Type** / **Available values** Existence / A <> 0 Coherence / A => B e A<=> B Congruence / A = B, A > B, A < B, A >= B, A<= B, A<> B
Dataset	Allows you to define the dataset or datasource on which to run the check.
Filter	Allows you to define an additional filter to limit the data on which to perform the check when running the rule in the DTP operation.
Deviation	(only with **Type** = **Congruence**) Allows you to define the maximum amount (as an absolute value) of the difference between the calculated values for the data related to *Set A* and *Set B*, in order for the check to be correct.
Label	Allows you to define a label under which to group a group of rules to be used in the DTP. The field is optional.
Condition fields	Allows you to select the dataset fields on which filters can be set. The field is optional.
Grouping fields	Allows you to select the dataset fields that define the tuple according to which to group the values to be checked.

Buttons in tab

Element	Description
	Adds a checking rule.
	Duplicates the selected checking rule.
	Deletes the selected checking rule.
	**Values**: opens a page to define the values on which to apply the check with the **operator** for the dataset fields selected as condition fields. See Values page.

Values page

 *Application database > Analytical Workspaces tile > workspace tile > Data transformation package tab > Checking rules > ⋮ > Values*

Purpose of the page

This page allows you to filter the values by the dataset fields used as condition fields.

Page sections

Element	Description
Set A	• (With coherence or congruence rules) Allows you to insert a value for the first term of the rule. • (With existence rule) Allows you to insert a value for the condition field.
Set B	(With coherence or congruence rules) Allows you to insert a value for the second term of the rule.

Page fields

Field	Description
Value	The value or formula for the value calculation. The records of the condition field are filtered based on these values. ***Note**: if no value is specified for the rule, all the values present in the data to be checked will be included.*

Calculation rules page

 *Application database > Analytical Workspaces tile > workspace tile > Data transformation package tab > Calculation rules*

 *Application database > Navigation panel > Analytic Information Hub > workspace tile > Data Transformation Package tab > ⊕ new DTP*

Purpose of the page

This page allows you to define calculation rules that make it possible to perform complex calculations, working on dataset rows. In contrast, standard DTP calculation operations process transformations in column mode. It is divided into two tabs: **Script** and **Logical dimensions**. In both tabs, it is possible to work using the "spreadsheet" function, which allows you to view and edit data, like in an Excel sheet.

Script attributes tab

Element	Description
Code	Code of the calculation script. Must not contain special characters. Follows the same rules as used for the application's code fields.
Description	Description for the various languages defined for the user.
Script	Script that defines the calculation rule.  *Expand the field to make the formula easier to read.*
Label	Unique code used to group rules. Can be used during the definition of the DTP operation, to run all rules with the same label.
Enable	If selected, it allows the rule to be used during the execution of a DTP. Selected by default.
Run condition	Allows you to enter, as an SQL formula, the condition to be checked in order to run the script. If not specified, the script is always run.
Start date	Allows you to specify whether the script has a start date.
End date	Allows you to specify whether the script has an end date.

IMPORTANT:

- If start and/or end dates are specified, they are considered against the range of the period scenario, which must be fully included within the validity period of the script. If not specified, the script is always run.

Logical dimensions attributes tab

Element	Description
Script operand code	Code that identifies the script operand. Must not contain special characters. Follows the same rules as used for the application's code fields.
Logical dimension	Logical dimension of the operand
Field code	Field of the dataset associated with the logical dimension of the operand.  ***IMPORTANT**: the Script operand code/Logical dimension pair must be unique to the Analytical Workspace.*

DTP activity cockpit

User

Application database > Processes tile > select process > Contributor > step tab > (Actions section) DTP > Activity cockpit

Navigation panel > Contribute > Process > select process > Contributor > step tab > (Actions section) DTP > Activity cockpit

Purpose of the page

This page shows the list of activities that can be run relating to the DTP associated with *Data Transformation Package* tasks present in the workflow: the information present on each row relates to the last execution.

Activities are displayed following the order of the DTPs present in the workflow, and therefore by level and code. For tasks run in parallel, sorting by task code is applied.

The *DTP* activity cockpit is active only where an entity and a step are selected.

Note: You can select only rows belonging to the same Analytical Workspace to run tasks. If activities belonging to different DTP are selected, the activities will be processed following the order of the DTPs

present in the workflow, therefore by level and code. For tasks run in parallel, sorting by task code is applied.

Column	Description
✔	Box to select the activity. Not selected by default.
AW - DTP	Shows the description of the AW and the description of the DTP to which that activity belongs.
Activity	Shows the code and the description of the activity. If this is part of a loop, the loop code is entered in round brackets before the activity code.
Dataset	Shows the description of the target dataset to which the activity is associated.
Type	Shows the description of the type assigned to the target dataset to which the activity is associated.
Start date	Shows the start date and time of the last time the activity was run.
End date	Shows the end date and time of the last time the activity was run.
User	Shows the user who last ran the activity.
Result	Shows the result of the last time the activity was run.
Data	Shows the icon to open the partition data details.

Icons and/or links on the page

Element	Description
	Indicates that the data processing related to the activity has been completed.
	Indicates that the data processing related to the activity is being processed.
	Indicates that it was not possible to complete the activity related to a *Data Transformation Package* task present in the workflow and associated with an Analytical Workspace DTP
	Indicates that that activity has not been run yet.
	Indicates that the data processing has been completed with at least one error. Click on the yellow traffic light to open a detail window showing the list of operations that generated that error during the last run, and the number of erroneous records found. If an operation with errors runs a check on a dataset, you can download a file containing the dataset rows containing errors. The first row of that file will contain the operation code and description and the message set for errors found during the check.

Buttons in the window

Element	Description
	Opens a pop-up to display the data contained in the Partition written by the specific activity. More details can be found on this page:Finestra Dettagli partizione.
	Update the page.
Run	This is enabled only when activities are selected. When clicked, the data processing of the selected activities starts.

Data processing is complete window

Application database > Processes tile > select process > Contributor > step tab > (Actions section) DTP > Activity cockpit > select activity > Run

Application database > Navbar > Processes & Workflows > Process > select process > Contributor > step tab > (Actions section) DTP > Activity cockpit > select activity > Run

Purpose of window

This window shows the outcome of the DTP data processing. In particular, it shows the outcome of all the activities and operations involved in the data processing.

Window structure

Element	Description
Data Processing is complete	Shows a list of data processing steps.
Management buttons	These allow you to manage sections of the window. • **Show only in case of error**: for subsequent data processing, the **Data processing is complete** window will only be shown in case of an error when running the data processing. • **Show/Hide log details**: shows the Log details section. • **Export**: exports the data on the running of the data processing (information on steps and log details) in XLSX format.
Detail log	(Can be viewed by clicking **Show log details**) Shows detailed information on the running of data processing: • **Details**: details of the data processing run • **Exceptions**: warnings and non-blocking errors • **Statistics**: statistical data related to the run • **SQL information** • **Notes**

DTP activities

Contents

This section covers the following topics:

(DTP) activity definition ... 20
DTP Debug Activity ... 21
Duplicate Activity window ... 23
Activity Debug Window ... 25
Temporary Debug Partitions ... 26
Temporary Debug Partitions page ... 26
Definition of activity and operation loops ... 27
Uploads ... 28
Loadings Page ... 28
Update data model ... 29
Update rules ... 31
Write on another dataset ... 33
Export to Financial Workspace ... 33
Operations ... 34
Operations ... 35
Matching fields page ... 40
Advanced settings page ... 43
Ask AI for DTP ... 43
Insert from other dataset ... 46
Enrichment from another dataset ... 47
Enrichment with formula ... 49
Enrichment with formula field ... 49
Enrichment with Reference dataset ... 50
Enrichment with ranking ... 53
Insert with reference dataset ... 54
Insert with pivot transposition ... 54
Insert with unpivot transposition ... 56
Join from another dataset ... 57
Analytical allocations ... 58
Allocation details page ... 60
Special method ... 61
Check with formula ... 62
Check with dictionary ... 62
Check with rule ... 64
Check missing keys ... 64
Existence check ... 66
Check multiple keys ... 67
Loading to Analytical Workspace ... 68
Delete with formula ... 69
Delete multiple keys ... 69
Delete missing keys ... 70
Insert with calculation rule ... 72

(DTP) activity definition

Application database > Analytical Workspaces tile > workspace tile > (Data Transformation Package tab) DTP tile

Application database > Navbar > Analytic Information hub > workspace tile >

Application Database > Navigation Panel > Analytic Information Hub > workspace tile > (Data transformation package tab) DTP tile

Purpose of the page

This page allows you to define DTP activities.

Page structure

Element	Description
List of activities	Shows the list of activities that make up the DTP and which will be run in alphanumeric order.
Activity properties	Allows you to manage the properties of the selected activity.

Activity properties

Element	Description
Activities	Activity identifier ***Note**: characters with ASCII codes from 33 to 126 are permitted, except for the following: 61, 124, 44, 39, 34, 47, 92, 58, 42, 63, 60 or 62.*
Description	Description of the activity in the active languages of the application.
Target dataset	Allows you to select the dataset on which to run the activity.
Typology	Specifies the type of the target dataset.
Empty Dataset	If selected, the content of the dataset/type will be deleted before the activity is run.
Update Reporting Layer	If selected, it deletes the content of the Reporting Layer at the end of the activity, overwriting the previous content.
Reporting layer filter	Allows you to insert an SQL formula to filter the deletion of Reporting Layer data.
Run condition	Allows you to enter, as an SQL formula, the condition to be checked in order to run the activity. If not specified, the activity is always run.
Level	Specifies the order in which to run the activity in the DTP. When several activities have the same level, the data processing simultaneously runs the activities which do not use the same resources.
Loop	Specifies the loop of which the activity is part. See "Definition of activity and operation loops" on page 27.
Executed distribution	(Visible, in the cloud, only if the environment has a specific system property and was enabled in the workspace creation step). If enabled, the system uses the distributed architecture to execute the activity, i.e. it runs the data processing outside of the application.

Buttons on the page

Element	Description
	**Operations**: Opens a page to define operations. See Operations. **Loadings**: Opens a page to define loadings. See Loadings.
	**Duplicate activity**: opens a window to create a new activity based on the selected activity. See Duplicate Activity. **Debug**: opens a menu with the following options: • **Run Debug**: opens a window to run a DTP in debug mode, using independent data partitions. See Debug Activity. • **Temporary debug partitions**: opens the page with the list of temporary partitions created by the DTP debug data processing. See temporary debug partitions. **Loop**: opens the activity and operation loops configuration window See Loop.

DTP Debug Activity

Introduction

DTP Debug Activity is used to partially execute a DTP without changing partition data.

To launch a DTP in debug mode, it is necessary to select an activity and enter the run parameters via the "Activity Debug Window" on page 25

When a DTP is debugged, the following options must be considered:

- **Debugging executes the selected activity and all lower-level activities (or loops)**.
- When there is a loop, the system derives the level of the loop from the lowest level, MIN(level), of all the activities belonging to the loop.
- Loading operations are not run.
- If there are parameters (AW or reporting) to be set up, a dedicated window will pop up to enter the required values and run the data processing. The audit window displays the name assigned to the target dataset at runtime.
- It is not possible to use datasets partitioned by Business Cycle & Entity in case of multi-organization process.
- For the operation field:
  - if no operation is selected and the field is left blank, debugging performs all operations of the task.
  - if an operation is selected, debugging performs that data processing up to and including that operation; subsequent operations will not be performed.

  ***IMPORTANT**: In the event that the selected operation is part of a loop of several operations, debugging executes the loop to which this operation belongs only up to the selected operation.*

Debugging of activities belonging to a loop

If the selected activity does not belong to the loop but to a higher level than the loop, the system runs the loop, including all activities belonging to it.

Example 1

Activity list

CODE ACTIVITY	LEVEL ACTIVITY	LOOP
ACT_001	0	
ACT_002	1	MY_LOOP_1
ACT_003	2	
ACT_004	3	MY_LOOP_1
ACT_005	3	

The selected activity is ACT_003 which has level 2,

Debug launch.

The loop MY_LOOP_1 has level 1 (because it takes it from the lowest level of the activities belonging to the loop).

The system runs all activities or loops with a level lower than the level of the selected activity (including also both activities of loop MY_LOOP_1):

then ACT_001, loop MY_LOOP_1 (ACT_002, ACT_004), ACT_003.

If the selected activity belongs to a loop, the system performs the following actions:

1. retrieves the loop level to which the selected activity belongs
2. determines the out-of-loop activities to be performed and runs them
3. performs all activities in the loop at a lower level than the selected activity in the loop.

Example 2

Activity list

CODE ACTIVITY	LEVEL ACTIVITY	LOOP
ACT_001	0	LOOP_X
ACT_002	1	LOOP_Z
ACT_003	2	LOOP_X
ACT_004	2	LOOP_Y
ACT_005	3	LOOP_Y
ACT_006	4	LOOP_Z
ACT_007	5	LOOP_Y
ACT_008	6	

Debugging operations belonging to a loop

If an operation is selected in the debug run window and the selected operation is part of a loop of several operations, debugging loops the operations up to and including that selected operation.

Subsequent operations are not performed.

Example 3

Activity list

CODE ACTIVITY	LEVEL ACTIVITY	LOOP
ACT_001	0	
ACT_002	2	
ACT_003	3	
ACT_004	3	

Task list of activity ACT_02:

CODE OP	LOOP
OP_001	
OP_002	**LOOP_X**
OP_003	**LOOP_X**
OP_004	**LOOP_X**
OP_005	

The selected activity is ACT_002 which has level 2.

Debug is launched by selecting operation OP_003.

OP_003 is part of LOOP_X, a loop of several operations since OP_002, OP_003 and OP_004 all have LOOP_X associated with them and are consecutive.

Since operation OP_003 was selected in the debug launch window, the system runs the LOOP_X loop, but only for operations OP_002 and OP_003.

- then OP_001, LOOP_X (OP_002, OP_003) are executed
- operation OP_004 is not executed even if it belongs to LOOP_X because it is subsequent to the operation selected in the debug window.
- OP_005 is not run for the same reason.

Duplicate Activity window

Application database > Analytical Workspaces tile > workspace tile > (Data Transformation Package tab) DTP tile > ✨ > Duplicate activity

Application database > Navbar > Analytic Information hub > workspace tile > (Data Transformation Package tab) DTP tile > ✨ > Duplicate activity

Application database > Navigation panel > Analytic Information Hub > workspace tile > (Data transformation package tab) DTP tile > > Duplicate activity

Purpose of window

This window allows you to launch a data processing that duplicates an existing task in the current DTP activities.

Duplication occurs with the following limitations and behaviour:

- In case of error, the entirety of the data processing will be cancelled.
- Operations which have references to dataset columns are only duplicated if the target dataset matches the source dataset.
- For *Loading to Analytical Workspace* operations, the operation is only copied if the target dataset matches the source dataset.
- For *Special Method* operations, the values are not copied.
- For *Allocation*, the details are not copied.
- The system does not support the duplication of the *Enrichment with formula field* operation or *Enrichment with reference table* operation field relationships.
- Fields containing filters or formulas such as *Target filter*, *SQL formula*, *Run condition* are copied without any changes, even if they contain references to columns of the target dataset.
- When loading a *Write on another dataset*, the loading will only be copied if the target dataset/type matches the source dataset/type.
- In case of upload > **Export to Financial Workspace**, the upload will be copied even if no Financial Workspace mapping is configured for the target dataset.
- In case of upload **Export Reporting layer to file**, the upload will only be copied if the target dataset/type pair matches the source dataset/type pair.
- Field relationships related to operations are duplicated if the target dataset matches the source dataset or if the target dataset contains a fields with the same code as a field of the source dataset.
- *If there are any fields that do not match and for which no relationship is created, use the Matching Fields Import/Export function after duplicating the operations.*
- The special values **N** (automatically match remaining fields) and **F** (apply formula to remaining fields) are copied in any case.
- A warning message is shown for every operation or loading that is not copied.

Fields in window

If an activity has been selected before this window is opened, the window fields (except **Activity code**) will be defined automatically with the same values as the selected activity. The values can be edited.

Element	Description
Source DTP	• Allows you to select the DTP containing the activity to be duplicated. The current DTP is selected by default.
Source activity	Allows you to select the activity to be duplicated.
Activity code	Allows you to enter the code of the new activity.
Description	Allows you to enter the description of the new activity.
Target dataset	Allows you to select the target dataset for the new activity. ***Note**: only online datasets can be selected.*
Typology	Allows you to select the target type for the new activity.

Buttons in the window

Element	Description
OK	Starts the data processing that duplicates the task with the relevant operations and uploads allowed. At the end of the data processing, an audit window shows the results of the data processing. In case of error, the entirety of the data processing is cancelled and the activity will not be duplicated.
Reverse	Closes the window without saving changes.

Activity Debug Window

Application Database > Analytical Workspace tile > workspace tile > (Data transformation package tab) DTP tile > > Debug > Start Debug

Application Database > Navigation Panel > Analytic Information Hub > workspace tile > (Data transformation package tab) DTP tile > > Debug > Start Debug

Purpose of window

This window allows you to set the parameters to launch a DTP in debug mode, using partitions of independent data.

Fields in window

Please find below the most important fields.

Element	Description
Process	indicates the DTP-related process to be run.
Scenarios	indicates the scenario on which the DTP is to be run.
Period	indicates the period on which the DTP is to be run.
Step	mandatory only in the case of a multiorganizational process; indicates the DTP phase to be performed.
Size	indicates the launch dimension (Entity or Analytical Dimension).
Update Reporting Layer	indicates whether the launch should also perform the update operations of the reporting layer.
Context independent run	indicates whether the launch must comply with the rules of the 'context-independent' mode. If the options is selected, the DTP will be able to find only partitions relevant to the launch and not other permanent partitions of the Calculation Layer; whereas, if the option is disabled, the DTP will find the partitions of a standard launch, namely the existing ones.
Operation	indicates the operation of the selected task at which debug execution is to end; if left blank, it is not considered a limit for the selected task.

Buttons in the window

Element	Description
OK	Starts the data processing that starts the debugging task with the relevant operations. At the end of the data processing, an audit window shows the results of the data processing. In the event of an error, the entire data processing is aborted and the debugging does not start.
Reverse	Closes the window without starting debugging.

Temporary Debug Partitions

Application Database > Analytical Workspace tile > workspace tile > (Data transformation package tab) DTP tile > > Debug > Start Debug > Temporary Debug Partitions

Introduction

The temporary debug partitions page displays the temporary partitions created by the DTP debug data processing (starting with DTP_%), see DTP debug.

IMPORTANT:

- Each time the DTP debug is started, the temporary partitions referring to that data processing are deleted, if any, and recreated with data from the current data processing.
- These tables will be deleted by the system after two days.

Temporary Partition Data Window

The button at the side of each partition opens the window with its **read-only** data.

This window shows the list of attributes contained in the partition and the values.

It is possible to download a csv file with the window data by enabling function **0004301** 'Analytical workspace - Data transformation package - Activity - Debug - Debug temporary partitions - Export CSV document' of the OWS_DATAFLOW form.

***NOTE**: User rights do not apply to data display.*

Buttons in the window

Element	Description
CSV	Allows you to export the partition data to a csv file.

Temporary Debug Partitions page

Application database > Analytical Workspaces tile > workspace tile > (Data Transformation Package tab) DTP tile

Application database > Navbar > Analytic Information hub > workspace tile >

Application Database > Analytical Workspace tile > workspace tile > (Data transformation package tab) DTP tile > > Debug > Start Debug > Temporary Debug Partitions

Purpose of the page

This page is read-only and displays all temporary partitions created by the DTP debug data processing.

Page structure

Element	Description
List of partitions	Shows the list of read-only partitions created by the DTP debug. See DTP debugger.

Activity properties

Element	Description
Partition table name	The name of the temporary partition table.
Dataset	The name of the dataset specified in the activity.
Typology	Type of dataset specified in the activity.
Date	Creation date of the partition.
Data	Info button to display partition data.

Definition of activity and operation loops

Application Database > Analytical Workspace > (Data transformation package tab) DTP tile > DTP tile > > Loop

Application Database > Navigation Panel > Analytic Information Hub > workspace tile > (Data transformation package tab) DTP tile > DTP tile > > Loop

Purpose of the page

This page allows you to define loops for the activities or operations of a DTP, in case you want to run them several times.

Loop Field Attributes Tab

In the Attributes tab, fields can be defined.

Field	Description
Loop Code	Mandatory, the loop code of one or more activities or operations of the DTP.
Description	Loop description, you will find all languages defined for that user. The description field 0 is mandatory.
List of values	Text field containing the query to retrieve the list of values.

IMPORTANT:

for the **List of Values** field:

- the maximum number of values the system can handle is 99.
- for CCH Tagetik to resolve an explicit list without generating an error, a query must be specified as in the examples below.
- one or more fields may be specified, for each field an alias may also be specified.

Example 1

```
select 'A'
UNION select 'B'
UNION select 'C';
```

Example 2

```
select 'January' as month, 'Monday' as day, 'JM' as cod
UNION select 'February' as month, 'Monday' as day, 'FM' as cod
UNION select 'March' as month, 'Monday' as day, 'MM' as cod;
```

WARNING:

To define the order of execution of the DTP activities, the system considers the **MIN(level)** of all activities belonging to the loop as the '**loop** level'. Regardless of the result of the run condition.

Whereas, there is no "**loop level**" for the operations loop as the operations have no level. In this case, loops of several operations can only contain consecutive operations, whereas if the same loop is referenced by non-consecutive operations, these operations will be run in separate loops.

Execution

- The activity loop always runs separate from any other parallel activities.
- Within the loop, all activities run in series. The order of execution is defined by the level (0 has higher priority) and with the same level it is not possible to determine the order of execution a priori.
- In the event that the query entered in the List of Values field does not deliver any results, the system does not execute the activities of that loop and the Loop information message is written in the audit, which was not run because that list is empty.
- The maximum number of operations that can be executed by a task containing loops of operations is 99.

Uploads

Introduction

Loadings are special operations for downloading processed data into the activity's target dataset/target. Loadings are carried out as the last step, once all of the activity operations and any reporting layer updates have finished.

Loadings are executed according to the order indicated in the **Sorting** field, which can be edited but must remain unique.

Write on another dataset

This allows you to report, on a dataset's Reporting Layer, the data of the current activity's target dataset/type. To do this, it uses the functionalities present in the Quick Data Loader.

Update data model

Populates a dimension based on the data present in a dataset. The source of the loading is the current activity's dataset/type, from which the operation is created.

Update rules

This data loading operation allows a rule to be populated from the data in the dataset.

Export Reporting layer data to file

Allows the Reporting Layer data of the target dataset of the current activity to be exported to file. To do so, it exploits the functionality present in Data Target Mapping: File. Currently only Data Target Mapping: Files exporting to Current Server or Endpoint are supported.

Export to Financial Workspace

Allows the Reporting Layer data of the target dataset of the current activity to be exported to the Financial Workspace. To do so, it exploits the functionality present in the Financial workspace mapping.

Loadings Page

Application database > Analytical Workspaces tile > workspace tile > (Data Transformation Package tab) DTP tile > ⋮ > Loadings

Application database > Navbar > Analytic Information hub > workspace tile > (Data Transformation Package tab) DTP tile > > Loadings

Purpose of the page

This page allows you to download processed data into the activity's target dataset/type.

Attributes

Element	Description
Loading Code	Loading code. Must be unique within the activity.
Loading type	Type of loading carried out. Update data model and load data onto another dataset are currently available. See Write on another dataset.
Description	Description in available languages

Update data model

Introduction

This data loading operation allows you to populate a dimension based on the data present in a dataset.

The source of the loading will be the current activity's dataset/type, while the target is the dimension selected in the loading operation configuration (see Data Target Mapping page (Update Data Model)).

> ***Note**: at the moment, only analytical dimensions can be selected, and it is not possible to add or edit the Reporting Languages or Custom Attributes.*

Data verification process

Before inserting or adding elements of a dimension, the data processing checks that the data present in the dataset are valid. To do this, it performs the following checks:

1. **checks on elements**: when the "Code" field and at least one other of the "Description", "Start date" and "End date" fields are mapped in the mapping.
2. **checks on relationships**: when the "Code", "Hierarchy" and "Node" fields are mapped in the mapping.

Examples

Example 1

In this example, a **"Data Target Mapping: update data model"** is configured with all the fields mapped:

Dataset

CODE	DESCRIPTION	HIERARCHY	NODE	START DATE	END DATE
001	DESC 001	H01	N01	13/02/2022	15/02/2023

Checks on the elements:

- there are no errors, the element with code 001 is added to the analytical dimension table if not already present, otherwise it is updated.

Checks on the relationships:

- if the hierarchy/node pair exists and there are no errors, element 001 is related to the specified hierarchy/node pair,
- if the hierarchy/node paid does not exist, the element is not related.

Dataset

CODE	DESCRIPTION	HIERARCHY	NODE	START DATE	END DATE
002	DESC 002	-	-	13/02/2022	15/02/2023

Checks on the elements:

- there are no errors, the element with code 002 is added to the analytical dimension table if not already present, otherwise it is updated.

Checks on the relationships:

- given that the hierarchy and node fields are mapped but not specified, an error is returned and the relationship cannot be inserted.

Dataset

CODE	DESCRIPTION	HIERARCHY	NODE	START DATE	END DATE
-	DESC 001	H01	N01	13/02/2022	15/02/2023

Checks on the elements:

- code field not specified error.

Checks on the relationships:

- code field not specified error.

Dataset

CODE	DESCRIPTION	HIERARCHY	NODE	START DATE	END DATE
003	DESC 003	H01	N01	13/02/2022	15/02/2023
003	DESC 004	H01	N01	13/02/2022	15/02/2023

Checks on the elements:

- two elements with the same code, the checks on elements check whether other fields also contain the same value. In this case, the description field does not have the same value, so a duplicate element code error is returned.

Checks on the relationships:

- these follow the same logic as the checks on the elements.

Dataset

CODE	DESCRIPTION	HIERARCHY	NODE	START DATE	END DATE
004	DESC 004	H01	N01	-	-
004	DESC 004	H02	N02	-	-

Checks on the elements:

- two elements with the same code, in this case all fields contain the same value, element 004 is added or updated.

Checks on the relationships:

- (with valid specified hierarchy/node pairs) two elements with the same code, in this case the hierarchy and node fields have different values, two relationships will be added or updated for each individual element.

Managing mandatory and non-mandatory fields

The **Update data model** operation takes account of all mapped fields. If a mapped field is mandatory, and no value is present in the dataset, the system reports an error both when the data processing inserts values and when a pre-existing element is updated. However, if the mapped field is not mandatory and the value does not exist for a certain row of the dataset, the system does not insert any value for new elements, whereas it overwrites the pre-existing value, thus emptying the field, when updating an element present in the analytical dimension.

Example 2

In this example, a **"Data Target Mapping: update data model"** is configured with all the fields mapped, considering that element 001 exists on the analytical dimension and has all fields defined.

- Dataset:

CODE	DESCRIPTION	HIERARCHY	NODE	START DATE	END DATE
001	DESC 001	H01	N01		

There are no errors, all fields of element 001 are updated. In particular, the “Start date” and “End date” fields are emptied, overwriting the previous value.

Execution conditions

Personal data option

To run this operation correctly when data protection is active, both the chosen analytical dimension and the dataset field, mapped with the description, must have the “**Personal Data Protection**” option active. Otherwise, the data processing will fail. At the end of the data processing, the descriptions will be protected for valid data inserted.

***Note**: not all personal data protection functionalities are managed; for example, the “Right to be forgotten” is not supported.*

Encoded option

The **“Encoded”** option must be selected on the field mapped with the description. If this condition is not met, the data processing will fail.

Update rules

Introduction

This data loading operation allows a rule to be populated from the data in the dataset.

***Note**: this operation is only available to ESG licence holders.*

The source of the upload will be the dataset/typology of the current task, while the target is the rule selected during the configuration of the upload operation (see Data target mapping (Update rules)).

***Note**: Currently only **'Calculation Logics'** can be selected.*

Data verification process

Before inserting or updating the elements of a rule, the data processing verifies that the data in the dataset are valid.

Script calculation logics, examples:

Example 1

In this example, a **'Data target mapping: update rules**' is configured to update the Script calculation logics with all mapped fields:

Dataset

CODE	DESCRIPTION	SCRIPT	LABEL	ACTIVE	RUN CONDITION	START DATE	END DATE
C001	DESC 001	TGKML 1.0	L001	1	ACTIVE = 1	15/09/2023	20/09/2023

There are no errors and the element with code C001 is inserted into the script type calculation logics if it is not already present, otherwise it is updated.

Dataset

CODE	DESCRIPTION	SCRIPT	LABEL	ACTIVE	RUN CONDITION	START DATE	END DATE
C002	DESC 002	TGKML 1.0	L002	0.5405	ACTIVE = 1	15/09/2023	20/09/2023

'Active' field error as the valid values are only 0 and 1.

Dataset

CODE	DESCRIPTION	SCRIPT	LABEL	ACTIVE	RUN CONDITION	START DATE	END DATE
C003	DESC 003	TGKML 1.0	L003	1.000	ACTIVE = 1	15/09/2023	20/09/2023
C003	DESC 003	TGKML 1.0	L003	1.000		15/09/2023	20/09/2023

Two identical lines, resulting in a duplicate code error.

Dataset

CODE	DESCRIPTION	SCRIPT	LABEL	ACTIVE	RUN CONDITION	START DATE	END DATE
C003	DESC 003	TGKML 1.0	L003	1		15/09/2023	20/09/2023
C003	DESC 003	TGKML 1.0	L004	0	ACTIVE = 1	15/09/2023	20/09/2023

Two elements with the same code, thus resulting in a duplicate code error.

Logical dimensions' calculation logics, examples:

Example 2

In this example, a **'Data target mapping:** update rules' is configured to update the Script calculation logics with all mapped fields:

Dataset

SCRIPT OPERAND CODE	LOGICAL DIMENSION	FIELD CODE
C0S_001	DL_001	CC_001
C0S_001	DL_002	CC_001

There is no error, the element COS_001/DL_001 and the element COS_001/DL_002 are inserted into the script type calculation logics if they are not already present, otherwise they are updated.

Dataset

SCRIPT OPERAND CODE	LOGICAL DIMENSION	CODE FIELD
C0S_001	DL_001	CC_001
C0S_001	DL_001	CC_001

Two elements with the same code, thus resulting in a duplicate code error.

***Note**: since the key consists of the fields '**script operand code**' and '**logical dimension**', it is currently not possible to update the '**logical dimension**' field of an existing element via the '**Update rule**' data processing.*

Managing mandatory and non-mandatory fields

The **Update rules** operation takes into account all mapped fields. If a mapped field is mandatory, and no value is present in the dataset, the system reports an error both when the data processing inserts values and when a pre-existing element is updated. However, if the mapped field is not mandatory and the value does not exist for a certain row of the dataset, the system does not insert any value for new elements, whereas it overwrites the pre-existing value, thus emptying the field, when updating an element present in the rule.

Example 2

In this example, a **'Data target mapping: update rules'** is configured with all fields mapped, assuming that element C001 exists on the Script type calculation logic and has all fields filled in.

- Dataset

CODE	DESCRIPTION	SCRIPT	LABEL	ACTIVE	RUN CONDITION	START DATE	END DATE
C001	DESC 001	TGKML 1.0	L001	1.000			

There are no errors, all fields of element C001 are updated. In particular, the **'Start Date'** and **'End Date'** fields are cleared, overwriting the previous value.

Execution conditions

Personal data option

The option **'Personal Data Protection'** is currently not supported in the calculation logics, so it must not be selected on the dataset fields. If selected, data processing fails.

Encoded option

The **"Encoded"** option must be selected on the field mapped with the description. If this condition is not met, the data processing will fail.

Write on another dataset

Introduction

This data loading operation uses the Load data from external datasources onto Analytical Workspace data processing as defined in a Quick Data Loader configuration.

The source of the loading is the current activity's dataset/type, while the target is one of the datasets/reporting layers of the current Analytical Workspace.

The deletion policies and editability checks defined for the importing of data via the Quick Data Loader are applied in the journal (see Import data via Quick Data Loader in the relative module).

For this type of loading, the following must be specified:

- the target Dataset for the downloading of data onto the reporting layer partition
- the Datasource mapping, i.e. the configuration to be used for loading the data.

Datasource mapping characteristics

The datasource mappings that can be selected in loading must have the following characteristics:

- read from the current activity's target dataset/type
- write on the selected dataset/reporting layer.

Parameters not supported

The following reporting parameters are not supported in the Multi Organisation Process:

- Numeric
- Dimension
- Simple Financial Statement Template
- Advanced Financial Statement Template

The following system parameters are not supported:

- #TARGET_PARTITION#

Export to Financial Workspace

Introduction

This upload operation allows the Reporting Layer data of the target dataset of the current activity to be exported to the Financial Workspace. To do so, it exploits the functionality present in the Financial workspace mapping.

Configuration

To add the upload to the configuration, simply select it as the Loading Type; the code and main description are pre-filled but can be edited before saving Once saved, you can always edit the descriptions, as with all uploads.

In the loading detail, a badge is shown indicating whether or not Financial workspace mapping is configured for the current task dataset. If this mapping is not available, you can still add and save the loading during the configuration phase.

Execution

Loading is performed within the activity if:

- the Financial workspace mapping is configured for the current activity dataset.
- DTP is launched by an Entity step.
- DTP is launched from a Custom Dimension step, and beta functionalities are enabled.

Loading is not performed within the activity if:

- the Financial workspace mapping is not configured for the current activity dataset.
- DTP is launched by an Analytical Dimension step.
- DTP is launched by a Custom Dimension step and beta functionalities are not enabled.

In these cases, the DTP will not be interrupted but will display a warning at the end of execution regarding the failed upload.

Operations

Introduction

The processing of a DTP activity may involve one or several operations, which the system runs in sequence. Every operation is the child of an activity and acts on a single dataset which is mandatorily indicated on the associated activity. Every operation performs calculations on a type of data in the target dataset associated with the activity. In the operation configurations, the operation can be subordinated to a certain SQL condition.

Operation types:

***IMPORTANT**: once an operation has been created, its type cannot be changed.*

Operation type	Description
Insertion	Allows you to add records to a dataset. Data the source of which is other datasets, datasources, reference tables etc. can be used as the source. For details on insertion operations, see: • Insert with reference dataset • Insert from other dataset • Insert with calculation rule • Insert with pivot transposition • Insert with unpivot transposition
Loading	Allows you to define one or several fields of a target dataset on an Analytical Workspace based on another source. For details on the loading operation, see: • Loading to Analytical Workspace

Operation type	Description
Enrichment	This allows you to update or define dataset columns that have already been defined. Other datasets, datasources, reference tables, SQL formulas, etc. can be used as sources. For details of enrichment operations, see: • Enrichment from another dataset • Enrichment with Reference dataset • Enrichment with formula • Enrichment with formula field • Enrichment with ranking • Join from another dataset
Allocation	Allows you to run an analytical allocation on a dataset using another dataset as the driver. For details on allocation operations, see: • Allocations
Check	Allows you to run validity checks on the records present in a dataset. There are several types of check, e.g. check with formula, check with dictionary. For details of checking operations, see: • Check with formula • Check with dictionary • Check with rule • Check missing keys • Existence check • Check multiple keys When a DTP is run from the contributor's cockpit and a lowest level element is selected, if the launch is of the standard type (not context-independent) and contains control operations that have encountered anomalies during execution, the DTP Report can be displayed. See "DTP reports" on page 4
Deletion	Allows you to perform deletions on a dataset. For details, see: • Delete with formula • Delete multiple keys • Delete missing keys
Special methods	Allows you to run customised operations based on an SQL script set up by the configurator. For details on special methods, see • Special Method

Operations

Application database > Analytical Workspaces tile > workspace tile > (Data Transformation Package tab) DTP tile > ⋮ > Operations

Application database > Navigation panel > Analytic Information Hub > workspace tile > (Data Transformation Package tab) DTP tile > ⋮ > Operations

Purpose of the page

This page allows you to create operations for DTP activities.

Page structure

Element	Description
List of operations	Shows the list of operations existing in the activity and allows you to change its execution order.
Activity information	Shows contextual information on the activity. • **Description**: description of the activity • **Dataset**: description of the target dataset • **Typology**: typology of the target dataset
Operation attributes	Shows specific properties for the selected operation

Attributes tab

The displayed fields will depend on the type of operation that has been selected.

Element	Description
Operation type	Type of operation to be run. Once defined, it can no longer be changed. For more details on operation types, see Operations.
Operation	Operation ID.
Description	Description of the operation in all system languages.
Execution conditions	Allows you to enter, with SQL syntax, the conditions to be checked in order to run the operation. If not specified, the operation will be performed automatically.
Source dataset	With operations such as **Insert from other dataset** / **Enrichment from other dataset** / **Insert with reference dataset** /**Join from other dataset** / **Check missing keys** / **Delete missing keys** / **Insert with unpivot transposition** / **Insert with pivot transposition** / Insert with calculation rules: • allows you to select the dataset or datasource from which to read the data. In case of **Insert from another dataset**/**Enrichment from another dataset**, it is not possible to select datasources which have a database other than the *current* database in their Connection fields.
Typology	With operations such as **Insert from other dataset** / **Enrichment from other dataset** / **Insert with reference dataset** / **Join from other dataset** / **Allocations** / **Check missing keys** / **Delete missing keys** / **Insert with unpivot transposition** / **Insert with pivot transposition** / **Insert with calculation rules**: • Allows you to select a source dataset typology. ***Note**: if a datasource is set as the source dataset, only the Reporting Layer typology can be selected.*
Source filter	With operations **Insert from other dataset** / **Enrichment from other dataset** / **Insert with reference dataset** / **Join from other dataset** / **Allocations** / **Check missing keys** / **Delete missing keys** / **Insert with unpivot transposition** / **Insert with pivot transposition** / **Insert with calculation rules**: • Allows you to insert an SQL filter to be applied to the dataset being read, to reduce the number of records considered as the operation's source.
Target filter	With operations **Enrichment from other dataset** / **Enrichment with formula** / **Enrichment with formula field** / **Enrichment with reference dataset** / **Join from other dataset** / **Delete multiple keys** / **Delete missing keys** / **Check with formula** / **Check with dictionary** / **Check with rule** / **Check missing keys** / **Checking existence** / **Check multiple keys**: • allows you to insert an SQL filter to reduce the data on which to run the operation.

Element	Description
SQL formula	With **Check with formula** operations: • allows you to insert the SQL formula to be verified to run the check. With **delete** with formula operations: • it is used to identify the data to be deleted. AW parameters can be used to make the operation parametric.
All fields with dictionaries	With Check with diction**ary operations:** • if active, checks all fields containing a dictionary.
Fields to check	With Check with diction**ary operations:** • allows you to select the fields associated with dictionaries to be checked.
Enrich only null values	With operations **Enrichment from other dataset** / **Enrichment with formula** / **Enrichment with formula field** / **Enrichment with reference dataset** / Join from other dataset: • When active, the operation only updates fields with empty values. ***Note**: rows containing formulas the results of which have null value will not be changed.*
Run row by row	With **enrichment from other dataset/enrichmentwith formu**la **operations, only on D**B HANA database: • Runs the dataset update with an Update instruction. ***Note**: can only be used for Hana DB at present.* ***Note**: cannot be used if there are aliases in the source formula of a field relationship.*
Target field	With **Enrichment with formula field** operations: • allows you to indicate the field to be defined through the formula. With **Enrichment with ranking** operations: • allows you to specify the field to be enriched with the rank value
Formula field	With **Enrichment with formula field** operations: • allows you to indicate the field containing a formula with which to define the target field.
Reference dataset	With operations **Enrichment with Reference dataset/Insert with reference dataset/Insert with calculation rules** : • allows you to insert an SQL filter to apply to the reference dataset. With **Check - existence** operations: • allows you to insert the reference dataset in which the existence fields and condition fields are indicated.
Reference dataset filter	With operations **Enrichment with reference dataset / Insert with reference dataset / Existence check / Insert with calculation rules:** • allows you to set an SQL filter with which to filter the reference dataset data.
Advanced settings	With **Enrichment with Reference dataset** operations: • allows you to insert ranges as condition column values. See Advanced settings page.
Dataset to be allocated	With **Allocations** type operations: • allows you to select the dataset containing the data to be allocated.
Driver dataset	With **Allocations** type operations. • allows you to select the dataset to be used as the driver.
Driver typology	With **Allocations** type operations. • allows you to select the driver dataset typology.

Element	Description
Details	With **Allocations** type operations. • allows you to insert information on the source and target of the data to be allocated and to define the allocation driver dataset. See Allocation details page.
Checking rule labels	With **Check with rule** operations: • allows you to select a label related to the checking rule of the target dataset or the datasource. Only checking rules containing the selected label are run. ***Note**: if not specified, the system runs all the rules compatibly with what is defined in **Checking rule type**.*
Checking rule type	With **Check with rule** operations: • allows you to select a checking rule type: ◦ **Coherence**: only runs Check - coherence rules. ◦ **Congruence:** only runs congruence check rules. ◦ **Existence**: only runs existence check rules. ◦ **All**: runs all checking rules. See Checking Rules.
Message for the user	With operations **Check with formula** / **Check with dictionary** / **Check with rule** / **Check missing keys** / **Check multiple keys**: • allows you to insert the message to be shown in the audit when the outcome of the check is negative. If not specified, the audit will show a default message.
Datasource Mapping	With **loading on Analytical Workspace** operations: • allows you to select a datasource mapping from which to obtain the operation's target fields.
Special method	With **special method** operations: • allows you to define a customised operation as an SQL script. ***Note**: a special method can have the same code in all the managed databases.* When defined, the **Special method parameters** tab will appear.
SQL sorting criterion	With **Delete multiple keys** operations: • allows you to define an SQL sorting criterion for the fields of the dataset being deleted **Note**: if not set, the entire list of user-defined fields in the target dataset will be used, applying an ascending sort order to each field.
Ranking type	With **Enrichment with ranking** operations: • allows you to select a type of ranking ◦ Differentiate sorting: corresponds to the ROW_NUMBER function. The ranking number is incremented for each record in the section that has the same values in the fields defined by the sorting criteria The rank value is initialised for each section. ◦ Keep sorting: corresponds to the DENSE_RANK function. The same ranking value is assigned to records within the same section that have identical values in the fields defined by the sorting criteria. The ranking value is increased for each section.
Sorting rule	With **Enrichment with ranking** operations: • allows you to specify the SQL sorting criterion

Element	Description
Fields Mapping	With operations **Insert from other dataset** / **Enrichment from other dataset** / **Enrichment with formula** / **Insert with reference dataset** / **Join from other dataset** / **Check missing keys** / **Delete missing keys** / **Insert with unpivot transposition** / **Insert with pivot transposition**: • allows you to define correspondence between the fields of a source dataset and a target dataset, to pilot the enrichment of the target dataset fields. See Matching fields page. ***Note**: with Insert from other dataset and insert with pivot transposition operations, it is not necessary to define the matching field.* ***Note**: with Insert with reference dataset, in the absence of field relationships, or in the presence of a "All others" field relationship, the target dataset fields are defined using the fields that have the same name and are present, in order, on the reference (attribute) dataset or source dataset.*
Key fields	With **delete multiple keys** / **Check multiple keys** operations: • allows you to define which fields of the dataset should form the key used to identify duplicate rows. See Matching Fields page. With **Enrichment with ranking** operations: • allows you to define which fields of the dataset should form the key used to identify the data sections to be sorted
Source pivot field	With **Insert** with Pivot transposition operations: • allows you to select the field of the source dataset from which the values to be transposed as target columns will be taken
Pivot value source field	With **Insert** with Pivot transposition operations: • allows you to select the field of the source dataset from which the values to be inserted as the contents of the transposed columns will be taken
Pivot value source field - grouping mode	With **Insert** with Pivot transposition operations: • allows you to select the aggregation function used for the pivot value source field
Pivot attribute target field	With **Insert with unpivot transposition** operations: • allows you to select the text type field of the target dataset where the aliases of the pivot-type matching field will be written
Pivot value target field	With **Insert with unpivot transposition** operations: • allows you to select the field of the target dataset where the values corresponding to the transposed column will be written.
Formula operand field	With **Insert with calculation rules** operations: • allows you to select the field in the source dataset corresponding to the column containing the operands to be used in the script formulas. ***Note**: Must have the same name on both the source and the target*
Value field	With **Insert with calculation rules** operations: • allows you to select the source dataset field corresponding to the column containing the values to be processed ***Note**: Must have the same name on both the source and the target*
Calculation rule	With **Insert with calculation rules** operations: • allows you to select the label associated with the calculation rules to be run. Given a source dataset and the logical dimensions table, the operation takes account of all scripts that are labelled as indicated in the operation. The result will be written into the target dataset indicated in the DTP activity.
Loop	Specifies the loop to which the operation belongs. See "Definition of activity and operation loops" on page 27.

Special method parameters tab

Element	Description
Special method parameters	These allow you to insert values for customised parameters defined in the special methods table (see Special methods page). If there are no customised parameters on the special method, the tab will remain empty.

Matching fields page

Application database > Analytical Workspaces tile > workspace tile > (Data Transformation Package tab) DTP tile > ⋮ > Operations > (Operation type) Matching fields

Application database > Navigation panel > Analytic Information Hub > workspace tile > (Data transformation package tab) DTP tile > ⋮ > Operations > (Operation type) Fields mapping

Purpose of the page

This page allows you to define correspondence between the fields of a source dataset and a target dataset, in order to guide the operation to insert or enrich fields of the target dataset.

For the **Enrichment with Reference dataset** operation, the window is accessed via the *Advanced settings* link.

For the **Delete Multiple Keys/Check Multiple Keys** *operations the mask can* be reached via the link **Key Fields**.

Page sections

Element	Description
List of relationships	Shows the list of relationships between fields of the source dataset and fields of the target dataset.
Attributes	Allows you to define the relationship properties, to pilot the operation.
Operation information	Shows contextual information on the operation: • **Code**: code of the operation for which the matching fields are been modified. • **Description**: operation description • **Type**: operation type

Relationship definition

Element	Description
Target field	With operations **Enrichment from other dataset**, **Enrichment with formula**, **Enrichment with ranking**, **Join with reference dataset**, **Enrichment with reference dataset**, Join from other dataset, Check missing keys, Delete missing keys, Insert with unpivot transposition, Insert with pivot transposition, Delete multiple keys, Check multiple keys • allow you to specify the field of the target dataset. With operations **Insert from other dataset**, **Insert with unpivot transposition**, **Insert with pivot transposition**, the system allows you to specify special values: • **Relate remaining fields automatically:** ◦ matches by name any fields not defined by the matching fields. • **Apply formula to the remaining fields:** ◦ applies the formula defined in the source field to any field not defined by other matching fields. **Note**: the two special options cannot both be saved for the same operation.
Condition	With **Enrichment with formula** operations: • allows you to specify the condition to be met in order to apply the SQL formula or the alternative SQL formula.
SQL formula	With **Enrichment with formula** operations: • allows the source value of the enrichment to be identified by means of an SQL formula, or by entering the code of the source field. It is possible to refer fields of the target dataset and the source dataset, via the aliases #DEST# and #SOURCE# For example, it is possible to write a formula as follows: CASE WHEN #DEST#.*CAMPOX* THEN #SOURCE#.*CAMPOY* ELSE #SOURCE#.*CAMPOZ* END so that you can customise a condition type matching field; in such a situation, it becomes mandatory to specify the datasets to which all matching fields defined via aliases belong, even in the case of a source formula with only one field. N.b. CAMPOX, CAMPOY, CAMPOZ indicate, by way of example, the dataset fields involved
Alternative SQL formula	With **Enrichment with formula** operations: • allows you to enter an SQL formula to be applied when the condition to apply the formula is not met.
Operator	With **Reference dataset enrichment** operations, allows you to select the operator to be applied in the comparison with the reference dataset. • >= • => • < • <=
Source	With operations **Insert from other dataset**, **Enrich from other dataset**, **Insert with formula**, **Enrichment with reference dataset**, **Join from other dataset**, **Check missing keys**, **Delete missing keys**, **Insert with unpivot transposition**, **Insert with pivot transposition**: • allows you to indicate the field in the source dataset from which the data is taken for enrichment. It can be defined with the dataset field code or an SQL formula to calculate its value.

Element	Description
Pivot value	With **Insert with unpivot transposition** operations: • allows you to indicate the field in the source dataset from which you will take the data for transposition that will be inserted as data in the pivot value target field specified on the operations page. It can be defined with the dataset field code or an SQL formula to calculate its value.
Pivot aliases	With **Insert with unpivot transposition** operations: • allows you to indicate the alias to be used as data in the Pivot attribute target field specified on the operations page.
Relationship type	With **Insert from other dataset, Join from other dataset operations :** • **Condition** indicates a join field. Condition fields allow you to identify reading records for every journal record. With **Insert from other dataset**, **Join from other dataset, Insert with Unpivot transposition** operations: • **Attribute** indicates that it is a field to be defined. With **Insertion with Unpivot transposition** operations: • **Pivot** indicates the field to be transposed
Grouping mode	Defines the aggregation function to be applied to the source fieldwhen reading data: • With **Relationship** set to **Condition**, the only permitted value is GROUP BY. • With **Relationship** set to **Attribute**, one of the following aggregation functions must be specified: SUM, MIN, MAX or COUNT. With **Enrichment** from other dataset operations: • **it is not possible to use aggregation functions** if an alias was used to refer to the target dataset or the source dataset. With operations **Insert from other dataset**, **Join from other dataset, Insert with pivot transposition**: • these restrictions do not apply. With **Insert with unpivot transposition** operations: • such restrictions do not apply. The only existing restriction is that when **Relationship** is set to **Pivot,** the GROUP BY value cannot be used.

Actions on this page

Element	Description
Export for Import	Runs the download of field relationship data in XLSX format.
Import from Excel	Opens a window to select an XLSX file from which to import data into the field relationships.

Advanced settings page

Application database > Analytical Workspaces tile > workspace tile > (Data Transformation Package tab) DTP tile > ⋮ > Operations > (Operation type) Advanced settings

Purpose of the page

This page allows you to insert ranges as condition column values. In particular, it allows you to specify the value with which to enrich a target dataset field if another field of the same dataset is less than, less than or equal to, greater than, or greater than or equal to the threshold values indicated in the source field of the reference dataset.

Page sections

Element	Description
List of relationships	Shows a list of the operation's conditions.
Attributes	Allows you to define the target field of the operation results and the ranges on the condition column values.
Operation information	Shows contextual information on the operation. • **Code**: operation code • **Description**: operation description • **Type**: operation type

Relationship definition

Element	Description
Target field	Allows you to select the target dataset field.
Operator	With **Reference dataset enrichment** operations, it allows you to select the allocation operator. • >= • => • < • <= These operators therefore define the relationship between the target dataset and the reference dataset.
Source	Allows you to indicate the source dataset field with which the enrichment will be run. It can be defined with a field code of the dataset or an SQL formula that calculates its value.

Ask AI for DTP

Introduction

Ask AI for DTP supports the administrator user by simplifying and speeding up the definition and maintenance of DTP.

In particular, Ask AI supports the administrator user in the following use cases:

- **Data model information**: identification of the Analytical Worksapace objects that can be manipulated by the DTP (dataset fields, parameters, functions, etc.).
- **From Text to SQL**: definition and maintenance of DTP SQL formulas (present in filters, execution conditions, matching fields, etc.).

***IMPORTANT**: The functionality is only supported for AWS customers; it is not supported for Azure customers.*

Synchronise DTP parameterization

The first step to ensure that Ask AI has all the necessary information to respond to the user is to synchronize the data in the Analytical Workspace.

This operation is performed automatically for specific changes: inserting, modifying and/or deleting metadata—such as a dataset field, a type, AW parameters, or DTP loops—thus allowing the user to work with Ask AI without having to worry about manually synchronizing the data.

***IMPORTANT**: it is necessary to perform this operation manually in all cases where the environment or Analytical Workaspce is imported or in the case of Disaster Recovery. (Application Database > Navigation Panel > Analytic Information Hub > action menu on workspace tile > Ask AI: sync DTP settings)*

Chat Ask AI

IMPORTANT: The Ask AI button is only visible if the user function **0004268 - DTP AskAi Assistant** is enabled.

When enabled, the button is only active in the DTP operation and matching fields pages. From both pages, you can use the chat to submit requests related to the 2 use cases: Data model information and From Text to SQL.

The user's request can be entered either manually or by voice command.

When the user's request, entered either manually or by voice command, is sent, the answer is displayed in the chat after being processed. For further details see Working with Chat Ask AI

IMPORTANT: the definition of the data model directly determines the quality of Ask AI chat responses.

When defining the Analytical Workspace, it is therefore important to avoid ambiguous objects, with non-intuitive descriptions, or which refer to system concepts (ex: avoid fields and descriptions referring to system keywords such as 'SOURCE' or 'TARGET').

Data model information

Through the Ask AI chat, the user can make enquiries about the definition of the data model of the Analytical Workspace they are working on, as well as about system parameters or F_ functions related to the Unified SQL syntax. See Unified SQL Syntax

The application context for the submitted requests is determined by the operation selected on the page.

Requests in the chat must always refer to the dataset for which information is being requested, using expressions such as 'from the source dataset', 'from the reference dataset', and so on. If the dataset is not specified in the request, the Ask AI chat will retrieve the information from the target dataset of the operation.

Example 1 - Questions on dataset definition

The user can ask questions about the fields in the target dataset of the activity to which the operation belongs, or about the source dataset or the reference dataset of the operation.

Questions of this kind can help when defining SQL filters or matching fields, without the need to open the dataset definition page for the analysis.

Element	Description
Give me the text type fields of the source dataset	the list of the first 10 field codes of text type, defined in the source dataset, separated by commas.
Give me the description of the FIELD_1 field	FIELD_1: field description 0 FIELD_1

IMPORTANT: Ask AI can work by similarity, attempting to retrieve the best match for the given request. This means that if the user makes a typo in the code — for example, entering FIEELD_1

instead of FIELD_1 — the system will still try to detect the best match and suggest the FIELD_1 field. **However, it is up to the user to validate the answer provided by the chat.**

Similarly, if other fields with similar code exist in the dataset, e.g. FIELD_11 or FIELD_12, the chat will provide a list of those fields.

Example 2 - Questions on AW parameters

The user can ask questions about the parameters defined at the Analytical Workspace level, without having to navigate to their definition page.

The user can refer to these parameters with expressions such as 'parameter AW', 'parameter @', or '@PARAM_NAME@'.

If...	...then
Give me the AW parameter param 4 (where param 4 represents part of the AW parameter description)	the list of the top 10 AW parameter codes that best match the user's request. For example, the answer could be like: @PARAM4@
Tell me whether the AW PARAM4 parameter must be requested at runtime	@PARAM4@: yes

Example 3 - Questions on system parameters

The user can ask questions about system parameters, referring to them with expressions such as 'system parameter', 'context parameter', 'parameter #', '#PARAM_NAME#'.

If...	...then
Give me the context parameter referring to the iteration value within an activity loop	'#ITERATION_VALUE#'
Explains what the #USER# parameter is for	'#USER#': user who launched the data processing **Note**: descriptions for system parameters and F_ functions will always be in English.

Example 4 - Questions on F_ functions

The user can make enquiries about the F_ functions used to write SQL formulas with unified syntax; see Unified SQL Syntax

If...	...then
Give me the unified syntax of the isnull	Ask AI returns: F_NVL(FIELD, VALUE)
Give me the translation of F_ ISNUMERIC for Postgres db	Ask AI returns: F_ISNUMERIC(expression): CASE WHEN %1$s ~ '^(-)?\d+(\.\d+)?(\,\d+)?$' is true THEN 1 ELSE 0 END
Give me the translation of F_SUBSTR	Ask AI returns the translation of the function in the syntax of the current DB vendor.

From Text to SQL

Through the Ask AI chat, the user can make requests on the definition of mathematical formulas, SQL formulas, filters, run conditions, combining dataset fields, F_ functions, AW and system parameters.

The user's request, expressed in natural language, is interpreted by the Ask AI chat, which retrieves the best matches among dataset codes, parameters, and functions.

The application context for the submitted requests is determined by the operation selected on the page.

Requests made through the chat must always refer to the dataset for which information is requested, using expressions such as 'from the source dataset', 'from the reference dataset', etc. If no dataset is specified in the request, the Ask AI chat will retrieve the information from the target dataset of the operation.

Example 5 - From Text to SQL

If...	...then
When the product is in the 'A' and 'B' list, then divide the price by 100 otherwise apply the discounted price	Ask AI returns: CASE WHEN PRODUCT IN ('A', 'B') THEN PRICE / 100 ELSE DISC_PRICE END Where the fields mentioned in the formula are the best matches identified by the chat, corresponding to the descriptions provided in the request. **Note:** Superscripts are required to indicate text strings to avoid ambiguity with any fields in the dataset.
Multiply the price by the difference between 1 and the discounted price	Ask AI returns: (PRICE * (1 - DISC_PRICE)) Where the fields mentioned in the formula are the best matches identified by the chat, corresponding to the descriptions provided in the request.
Apply condition when parameter # scenario contains 2025	Ask AI returns:F_INSTR('#SCENARIOS#', '2025') > 0

Note: the system always tries to construct formulas using a unified syntax, ensuring that the formula is independent of the current database. If the user wishes to use the syntax specific to the database in use, they must indicate in the request that F_ functions should not be used (e.g.: 'Apply condition when parameter # scenario contains 2025; do not use F_').

IMPORTANT: For all the above-mentioned use cases, if the Ask AI chat fails to identify the objects referenced by the user, it will provide an explanatory response indicating which information is missing.

Example 6 - Information Not Found

If...	...then
Give me the description of the FIELD_1 field of the source dataset	If the mentioned field is not available in the source dataset, Ask AI returns a message such as: The field 'FIELD_1' is not present in the available data. **Note:** the answer will be given in the same language as the question.

Error Management for Ask AI

When using Ask AI, there may be instances when a response cannot be obtained.
In such cases, error messages are displayed to help better understand the problem that has occurred.

Example 7 - Error Messages

If...	...then the system displays the error message:
the DTP project for Ask AI has not yet been populated with data from CCH Tagetik	"The data used in the DTP parameterization are not synchronized. Please perform synchronization to continue using Ask AI"

Insert from other dataset

Function and purpose of the operation

The **Insert from another dataset** operation allows you to populate one or more fields in a target dataset using values from a source dataset or calculated values.

In particular, it adds the rows present on the source dataset to the target dataset based on attribute-type field relationships shared between the two datasets.

If no matching fields are configured, the system automatically checks for possible matches based on the column names of the two datasets.

Advanced Tab (only visible for physical or virtual datasets)

The **source scenario** /period fields allow you to define a scenario/period from which to read data for the given source dataset. It is possible to specify a value using the AW parameters or through the same syntax used in the definition of cell fields. If the field is left blank, the DTP launch scenario/period is taken into account.

If a physical dataset is used and the specified scenario/period does not exist or is not active, the DTP proceeds without executing the operation, logging the missing execution in the audit.

Example

Source Dataset

Category	Product	Price
Electronics	Laptop	1329
Electronics	Bluetooth Speaker	99
Electronics	Smartphone	800
Toys	Puzzle Game	15

The source dataset has the columns Category, Product, and Price, while the target dataset only has Product and Price. Without explicit Matching fields, the system can match the two columns because they have the same names. Then the data processing copies the rows of these columns of the source dataset into the target dataset

Target Dataset

Product	Price
Laptop	1329
Bluetooth Speaker	99
Smartphone	800
Puzzle Game	15

Enrichment from another dataset

Function

The *Enrichment from another dataset* operation allows one or more fields in a target dataset to be populated using values from a source dataset or using calculated values. In particular, it modifies the value of existing fields in the target dataset.

At the end of this operation, the system checks whether several identical rows have been created, e.g. as a result of checking several conditions on the same row. If so, it returns an error.
Matching fields are mandatory because you must specify at least one condition column and one attribute column to enable the update.

Note: If both source and target have a null value for a column used as a condition, the system treats them as two different rows. Therefore, the condition is not satisfied.

At the end of the operation, a check is carried out to ensure that no rows have been duplicated during the enrichment (e.g. due to more than one condition being checked for the same row).

Note: On the operation page, the **Enrich only null values** flag has two options:

- selected: if the target dataset field set as an attribute in the matching fields is empty then it performs the enrichment, otherwise it does nothing.
- not selected: the target dataset field set as an attribute in the matching fields is always updated with the value of the source dataset

Advanced Tab

The **source scenario** /period fields allow you to define a scenario/period from which to read data for the given source dataset. It is possible to specify a value using the AW parameters or through the same syntax used in the definition of cell fields.

When the field is left blank, the DTP launch scenario/period is taken into account; however, in the case of a non-existent or inactive scenario/period, the DTP will go ahead without running the operation, reporting the failed execution in the audit.

Note: the advanced tab is only visible for physical or virtual datasets.

Example

This example shows the execution of an enrichment operation from another dataset.

Target Dataset

CONTRACT	CUSTOMER	AMOUNT	COUNTRY	RATE
C2	Madison White	5000	Rome	
C1	Joe Black	10000	New York	

Source Dataset

COUNTRY	RATE
Rome	1.7
New York	1.5

The goal is to update the RATE field of the target dataset with the value of the RATE field of the source dataset. In matching fields, the field to be used as a condition and the field to be used as an attribute are specified.

Matching fields

Target field	Source field	Relationship type
COUNTRY	COUNTRY	Condition
RATE	RATE	Attribute

After performing the *Enrichment from another dataset* operation , the resulting dataset will be as follows:

Target Dataset

CONTRACT	CUSTOMER	AMOUNT	COUNTRY	RATE
C2	Madison White	5000	Rome	1.7
C1	Joe Black	10000	New York	1.5

Example

Target Dataset

CONTRACT	CUSTOMER	AMOUNT	COUNTRY	RATE
C2	Madison White	5000	Rome	
C1	Joe Black	10000	New York	

Source Dataset

COUNTRY	RATE
Rome	1.7
Rome	2.0
New York	1.5

The goal is to update the RATE field of the target dataset with the value of the RATE field of the source dataset. In matching fields, the field to be used as a condition and the field to be used as an attribute are specified.

Matching fields

Target field	Source field	Relationship type
COUNTRY	COUNTRY	Condition
RATE	RATE	Attribute

Data processing fails because there are two rows in the source dataset that meet the condition COUNTRY = Rome and the system does not know which RATE to use to update the row in the target dataset.

Enrichment with formula

Function

The *Enrichment with formula* operation allows you to define the fields of a dataset using SQL formulas. In addition to the formula, it is possible to specify a condition to be checked in order to apply the formula.

If the condition...	Then...
is checked	the system will apply the formula defined in the *SQL formula* field. This formula is also applied if no condition is defined.
is not checked	the system will apply the formula defined in the *Alternative SQL formula* field.

Syntax in formulas

Recursive formulas can be defined within the same operation, by inserting the reference to another target field in the *SQL formula* field (or the *Alternative SQL formula* field), using the syntax *@destination_field@*. In this case, the system replaces the expression *@destination_field@* with the field relationship present in the target field *destination_field*.

That syntax cannot be used to refer to a target field if no explicit relationship with the source field has been defined in the target field. It is always possible to cite the *destination_field* as the table field. To do this, simply omit the special character @.

In case of circular references in the definition of target fields, the system shows a message with details of the fields involved. In case of Analytical Workspace parameters and target fields with the same name in the dataset, the Analytical Workspace parameters prevail.

Enrichment with formula field

Function

The *Enrichment with formula field* operation allows you to enrich a field of a target dataset using an SQL formula defined on a *formula field* present on the same row as the field to be enriched.

It is not possible to run updates with formulas containing keywords corresponding to commands such as DROP, UPDATE, and so on, or special characters that indicate the opening of a comment or the closing of an SQL statement.

Rows containing a formula with *null* value will not be edited.

Example

This example considers the following target dataset:

Formula	Destination	Field A	Field B
Field A + Field B		A	B
length(Field A + Field B)		A	B

The operation is set up so that the formula in the *Formula* field is applied to define the content of the *Target* field. Once the operation has been run, the dataset will contain the following values:

Formula	Destination	Field A	Field B
Field A + Field B	AB	A	B
length(Field A + Field B)	2	A	B

The first row of the target field contains the concatenation of the values contained in *Field A* and *Field B* given that the first row of the *Formula* field specifies the formula *Field A + Field B*.

However, after having concatenated the values of *Field A* and *Field B* (A and B, respectively), the second row calculates the length of the result, given that the *Formula* field of the second row contains the formula *length(Field A + Field B)*.

Enrichment with Reference dataset

Function and purpose of the operation

The **Enrichment with Reference dataset** operation allows you to enrich a target dataset using a reference dataset. In particular, the enrichment operation transfers the values of the attribute fields of a reference dataset, which acts as the source, to the columns of a target dataset. This is only possible when the datasets contain columns with the same field code and the same values.

It is possible to set the fields of the reference dataset as condition fields. If a condition field of the reference dataset is not specified, every value in the fields of the source dataset which corresponds to the target dataset makes the condition true.

Enrichment operations on ranges of values

On non-*Advanced settings* reference datasets (see Reference dataset), it is possible to define *advanced settings* for enrichment operations. These allow you to enrich the data, considering ranges of records in relation to a reference value. To do this, the following elements must be defined in the DTP operation settings:

- Target dataset field on which the enrichment operations will be performed
- A operator detects whether another field of the same target dataset is less than, less than or equal to, greater than, or greater than or equal to limit values indicated in the source field of the reference dataset.
- Source field of the reference dataset indicated with the code of the field or an SQL formula to calculate its value.

Open ranges can only be defined for data or number fields. For string fields, however, it is necessary to insert one condition to indicate the lower limit and one condition to indicate the upper limit of the range.

The system considers such advanced settings to be exceptions to the standard functioning of the enrichment operation, which is applied to all the other dataset fields.

***Note**: the % character has no specific function in advanced conditions and is treated as any other character.*

These advanced settings are valid as exceptions to the standard functioning of the enrichment operation.

***Note**: If both reference and target have a null value for a column used as a condition, the system treats them as two different rows. Therefore, the condition is not satisfied.*

Conditions with special characters

Special conditions can be used on the fields of the reference dataset by inserting values followed by the % character. For example, *A%* corresponds to all the elements that start with A. In this case, instead of correspondence between the values, the system uses the condition defined by the SQL operator 'like'. It is also possible for the system to check several conditions on the same row, generating new rows and enriching the existing rows. When the operation generates rows, the system shows a row duplication error message and details of the first 10 excess rows in the audit.

Example of how the operation works

This example shows how an operation to enrich a target dataset using a reference dataset works.

The target dataset will show the following details.

Column 1	Column 2	Column 3	Different Column
A	1		
B	2		
C	3		

The reference dataset will show the following details.

Column 1 (Condition)	Column 2 (Condition)	Column 3 (Attribute)	Column 4 (Attribute)
A	1	Result 1	Example 1
B	2	Result 2	Example 2

At the end of the enrichment operation, the target dataset will show the following details.

Column 1	Column 2	Column 3	Different Column
A	1	Result 1	
B	2	Result 2	
C	3		

***Note**: if the reference dataset had "%" reported in the condition column Column 1 for the first row, this would check the condition for all the values of that column on the target dataset, and therefore both the A values and the B values, bringing the total number of rows generated from two to three. Two of the generated rows would therefore be generated by checking the first condition, and one would be generated by checking the second condition. This causes an error on the operation that uses it, due to the duplicate rows generated by a condition. However, no problem would be detected for deploy type datasets (see Reference dataset - Deploy).*

Exceptions management (Level type field)

A reference dataset field can be configured as a *Level*. This type is useful for having a column of the dataset in which a level of exception to a behaviour defined in the reference dataset is specified.

***Note**: The column of type LEVEL (if present) cannot have null values.*

Example of how a level type field works

This example shows how an operation to enrich a target dataset using a reference dataset works, in the presence of a level type field on the reference dataset.

The reference dataset will show the following details.

COND_1	COND_2	ATTRIBUTE	LEVEL
AA	%	default A	0
AA	X	new value	1
B	W	default B	0

The target dataset will show the following details.

COND_1	COND_2	ATTRIBUTE
AA	X	
AA	Y	
AA	Z	
B	W	
C	W	

At the end of the enrichment operation, the target dataset will show the following details.

COND_1	COND_2	ATTRIBUTE
AA	X	new value
AA	Y	default A
AA	Z	default A
B	W	default B
C	W	

***Note**: the condition to be applied on which the higher level is set overwrites the condition defined with the previous level.*

Filters management (Filters type field)

A reference dataset field can be configured as a *Filter*. This type is useful for having a dataset column in which a filter to be applied to the target dataset in the run update step due to the same row is specified.

Note: The *FILTER* type column (if present) may also have *null* values, in which case the enrichment is not filtered.

Example of how a filter type field works

This example shows how an operation to enrich a target dataset using a reference dataset works, in the presence of a filter type field on the reference dataset.

The reference dataset will show the following details.

COND_1	COND_2	ATTRIBUTE	FILTER
AA	%	default A	CAMPO_INFO = '1'
AA	X	new value	CAMPO_INFO = '0'
B	W	default B	COND_1 = 'B'

The target dataset will show the following details.

COND_1	COND_2	CAMPO_INFO
AA	X	1
AA	Y	0
AA	Z	0
B	X	AA
B	W	B

At the end of the enrichment operation, the target dataset will show the following details.

COND_1	COND_2	ATTRIBUTE
AA	X	default A
AA	Y	
AA	Z	
B	W	default B
C	W	

Enrichment with ranking

Function

The *Enrichment with Ranking* operation allows you to enrich a field in a target dataset with a rank value calculated within sections defined by one or more attributes. The records will be sorted according to the chosen criterion.

On the transaction definition page, you must specify:

- Target Field: the field which is enriched with ranking values.
- Ranking type: allows you to select a type of ranking:
  - Differentiate sorting: corresponds to the ROW_NUMBER function. The ranking number is incremented for each record in the section that has the same values in the fields defined by the sorting criteria The rank value is initialised for each section.
  - Keep sorting: corresponds to the DENSE_RANK function. The same ranking value is assigned to records within the same section that have identical values in the fields defined by the sorting criteria. The ranking value is increased for each section.
- Sorting rules: free field where you can specify the sorting criteria of the operation.

***Note**: If no key fields are specified, the system will only apply the sorting criteria to calculate the ranking.*

***Note**: If the sort criteria are not specified, the system will sort the data using all custom fields, following the natural sorting of the table, excluding the target field. If the target field is to be included, the user must add it manually.*

***Note**: If a filter is used in the operation on the dataset, enrichment and sorting will only be applied to the data selected by the filter.*

Example

Category	Product	Price	Rank
X	AA	100	1
X	B	100	2
X	C	200	3
Y	D	200	1
Y	E	200	2
Y	F	300	3

Target field: Ranking
Ranking type: Differentiate sorting
Sorting criteria: Price
Key fields: Category

Insert with reference dataset

Function

The *Insert with reference dataset* operation allows you to insert values on a target dataset based on a source dataset. The transformation of the data is piloted and enriched by the reference datasets (see Reference dataset).

It is possible to set the values of the fields of the reference dataset as condition fields. If a value of a condition field of the reference dataset is not specified, every value in the corresponding field on the target dataset makes the condition true.

This operation does not take account of the management of the level field on the reference and does not involve checks on any rows to be duplicated, regardless of the type of reference dataset.

Insert with pivot transposition

Function and purpose of the operation

The **insert with pivot transpose** operation adds as many columns to the Target dataset as there are values to transpose.

On the operation definition page, you must specify the fields of the source dataset:

- Pivot attribute field: the source field from which the data to be transformed into target fields will be extracted
- Pivot value field: the source field from which data will be taken to be aggregated, by means of an aggregation function to be specified into the fields extracted by processing the pivot attribute field.
- Aggregation function: aggregation function for the pivot value field.

***Note**: the target field of a matching field cannot coincide with a pivot target field*

***Note**: pivot attributes that are not present in the target dataset will be ignored.*

Advanced Tab

The **source scenario** /period fields allow you to define a scenario/period from which to read data for the given source dataset. It is possible to specify a value using the AW parameters or through the same syntax used in the definition of cell fields.

When the field is left blank, the DTP launch scenario/period is taken into account; however, in the case of a non-existent or inactive scenario/period, the DTP will go ahead without running the operation, reporting the failed execution in the audit.

***Note**: the advanced tab is only visible for physical or virtual datasets.*

Example 1: base case without matching fields

Source Dataset

PRODUCT	CLIENT	COD_VALUE	VALUE
P1	C1	QUANTITY_03	10
P1	C1	QUANTITY_06	20
P1	C1	QUANTITY_09	30
P1	C1	QUANTITY_12	40
P2	C1	QUANTITY_03	50
P2	C1	QUANTITY_06	60
P2	C1	QUANTITY_09	70
P2	C1	QUANTITY_12	80

On the operations page, the following items are defined:

- Source pivot field: COD_VALUE
- Pivot value source field: VALUE
- Pivot value source field - grouping mode: SUM

Target Dataset

QUANTITY_03	QUANTITY_06	QUANTITY_09	QUANTITY_12
60	80	100	120

If there are no matches for a tuple for a given transposed column, no value will be displayed. See example 2 **tuple (2025 | Q2) transposed column NORTH**.

Example 2: Case with matching fields

Source Dataset

YEAR	QUARTER	REGION	SALES
2024	Q1	NORTH	200
2024	Q1	SOUTH	400
2024	Q1	NORTH	300
2024	Q2	NORTH	700
2025	Q1	SOUTH	800
2025	Q1	NORTH	500
2025	Q1	SOUTH	500
2025	Q2	SOUTH	300

On the operations page, the following items are defined:

- Source pivot field: REGION
- Pivot value source field: SALES
- Pivot value source field - grouping mode: SUM

Matching fields

Target field	Source	Grouping mode
YEAR	YEAR	GROUP BY
QUARTER	QUARTER	GROUP BY
GAIN	GAIN	SUM

Target Dataset

YEAR	QUARTER	NORTH	SOUTH	GAIN
2024	Q1	500	400	600
2024	Q2	700		500
2025	Q1	500	1300	1100
2025	Q2		300	200

Insert with unpivot transposition

Function

The *Insert with unpivot transposition* operation allows you to add as many rows in the target dataset as there are columns in the source dataset to be transposed, as defined on the matching fields page.

On the operation definition page, you must specify the fields of the target dataset to be associated with the pivot attribute and the pivot value, which must obviously be separate fields.

It is mandatory to define matching fields, as at least one pivot field must be specified, along with its alias

Note: Field values that are not listed on the matching fields page will not be brought into the target table.

Advanced tab

The **source scenario** /period fields allow you to define a scenario/period from which to read data for the given source dataset. It is possible to specify a value using the AW parameters or through the same syntax used in the definition of cell fields.

When the field is left blank, the DTP launch scenario/period is taken into account; however, in the case of a non-existent or inactive scenario/period, the DTP will go ahead without running the operation, reporting the failed execution in the audit.

Example 1: basic case: one attribute, one pivot and one alias

Source Dataset

PRODUCT	CLIENT	JUNE	DECEMBER
Prod 01	Client 01	15	30
Prod 02	Client 02	18	9

Matching Fields Table

Relationship type	Details
Attribute	Target field: PRODUCT, Source: PRODUCT
Pivot	Pivot value: JUNE, Pivot alias: PERIOD_JUNE
Pivot	Pivot value: DECEMBER, Pivot alias: PERIOD_DECEMBER

In operation page are defined:

- Pivot Attribute Fields: ATTRIBUTE
- Pivot Value Fields: VALUE

Target Dataset

PRODUCT	ATTRIBUTE	VALUE
Prod 01	PERIOD_JUNE	15
Prod 01	PERIOD_DECEMBER	30
Prod 02	PERIOD_JUNE	18
Prod 02	PERIOD_DECEMBER	9

Join from another dataset

Function and purpose of the operation

The *Join from another dataset* operation allows you to define one or several fields of a target dataset based on values present in a source dataset or calculated values. However, it does not run checks on any row duplication at the end of the operation. Thus, the number of rows produced varies depending on the indicated conditions.

Matching fields are mandatory because you must specify at least one condition column and one attribute column to enable the update.

***Note**: If both source and target have a null value for a column used as a condition, the system treats them as two different rows. Therefore, the condition is not satisfied.*

Advanced Tab

The **source scenario** /period fields allow you to define a scenario/period from which to read data for the given source dataset. It is possible to specify a value using the AW parameters or through the same syntax used in the definition of cell fields.

When the field is left blank, the DTP launch scenario/period is taken into account; however, in the case of a non-existent or inactive scenario/period, the DTP will go ahead without running the operation, reporting the failed execution in the audit.

***Note**: the advanced tab is only visible for physical or virtual datasets.*

Example

Target Dataset

CONTRACT	CUSTOMER	AMOUNT	COUNTRY	RATE
C2	Madison White	5000	Rome	
C1	Joe Black	10000	New York	

Source Dataset

COUNTRY	RATE
Rome	1.7
Rome	2.3
New York	1.5

In matching fields, the field to be used as a condition and the field to be used as an attribute are specified.

Matching fields

Target field	Source field	Relationship type
COUNTRY	COUNTRY	Condition
RATE	RATE	Attribute

After performing the *Join* operation *from another dataset* , the resulting dataset will be as follows:

Target Dataset

CONTRACT	CUSTOMER	AMOUNT	COUNTRY	RATE
C2	Madison White	5000	Rome	1.7
C2	Madison White	5000	Rome	2.3
C1	Joe Black	10000	New York	1.5

Analytical allocations

Function

The *Allocations* operation allows you to distribute an amount originally present on an individual item among several items, in order to itemise it. In particular, allocation allows you to distribute a value present on an item of a source dataset dimension among one or several dimensional fields of the target dataset.

For example, the cost of a service can be distributed across departments that have benefited from it by taking account of the products that they ordered.

To perform this distribution, the source dataset/type of dataset and driver dataset must be defined.

The driver dataset determines the detailed allocation coefficient for one or several dimensions to which it is possible to allocate. Any dimensional field can be used as the allocation dimension, except for Scenario, Period, Entity, Currency and Var. - Detail type.

It is also possible to define SQL filters to be applied to the data to be allocated.

Allocation details

To define an allocation operation, the data to be allocated and its target must be defined. A single operation can contain several details. This allows you to defined different methods for allocating various portions of the dataset to be allocated. Every detail deals with different portions of data in terms of the data to be allocated. If this does not occur, the system will generate duplicates.

The details are processed simultaneously. When several details share the same data tuple, the corresponding amount is allocated according to the information present on all details that are applied. This causes the amounts to be duplicated.

Allocated data generation conditions and restrictions Allocated data can be generated under the following conditions:

- The target dataset/type of the results of the operation has been defined on the activity.
- The corresponding source dataset/type, target and driver are different.
- For every source dataset field containing the data (measure) to be allocated, there is a numeric field with the same name on the target dataset.
- For every driver dimension on which data is being allocated, the target dataset has a dimensional field linked to the same dimension as the driver on which the data is being allocated (the code may be different).

Allocated data generation process

The allocated data is generated according to the following process:

Step	Description
1	The allocated amount is inserted into the measure (numeric field) of the activity's output dataset with the same name as the measure to be allocated from the dataset to be treated.
2	The value of the receiving dimensions is recovered by the driver and stored in a field in the output dataset. This field may not have the same name but must its type must be dimensional. The dimension must be the same as the driver allocation dimension.

Step	Description
3	Every other dimensional field present on the output dataset is populated by reading the field of the dataset to be treated, which has the same dimensional type, if it exists.
4	Every other non-dimensional field present on the output dataset is populated by reading the field of the same name in the dataset to be processed, if it exists.

Particular details of use cases for allocation

- To allocate separate portions of the same source dataset in different ways, you can define a single operation with several details.
- To allocate data that has already been allocated, it is necessary to create two different activities, in sequence, each containing one operation. The operation present in the second activity will have the target dataset/type of the previous activity as the dataset/type to be allocated.
- Activities containing allocation operations can be configured to be run in parallel, like any other activity. However, logical dependencies which prevent the various activities from running in parallel must not exist.
- Once any SQL filter present on the operation has been applied, the amount to be allocated is reported on the target dataset. If the treatment of part of the amount to be allocated is not specified in the definition of the details, the non-allocated data are reported in the target dataset without undergoing transformations.

Example

This example shows the allocation operation for the following data on the Account, Cost Centre and Product dimensions:

Dato da trattare:

Conto	Centro di Costo	Prodotto	Importo
PL00001	C01	P01	10
PL00001	C05	P02	20

Driver:

Conto	Centro di Costo	Prodotto	Importo
DRV1	C01	P01	1
DRV1	C01	P02	2
DRV1	C02	P02	3
DRV1	C03	P02	4
DRV1	C04	P02	5
		Totale	15

In this case, the calculation performed by the system for the *Cost Centre* custom dimension changes depending on the partialisation of the specified driver (the *Other filters based on data to allocate* parameter in the operations details). The possible options are listed below.

- No filter: every row of the data to be processed uses every row of the driver to allocate.

  Scritture generate:

Conto	Centro di Costo	Prodotto	Importo
PL00001	C01	P01	(10+20)*1/15 = 2
PL00001	C01	P02	(10+20)*2/15 = 4
PL00001	C02	P02	(10+20)*3/15 = 6
PL00001	C03	P02	(10+20)*4/15 = 8
PL00001	C04	P02	(10+20)*5/15 = 10

- Filter on lowest level elements: every row of the data to be processed uses the driver rows

defined on the same cost centre.

- Scritture generate:

Conto	Centro di Costo	Prodotto	Importo
PL00001	C01	P01	10*1/3 = 3.33
PL00001	C01	P02	10*2/3 = 6.66

In this example, the second row of the data to be processed was not allocated because the driver has no rows on C05. The driver rows defined on cost centres other than C01 were not used.

Allocation details page

Application database > Analytical Workspaces tile > workspace tile > (Data Transformation Package tab) DTP tile > Properties panel > (Operation type) select Allocations > Details

Purpose of the page

This page allows you to define the details of the allocation, or define the data to be allocated, the fields to which it will be allocated and the driver dataset to be used to pilot the allocation.

Page sections

Element	Description
List of details	Shows the details of the allocation: every row corresponds to data to be allocated.
Allocation details tabs	These allow you to specify the field containing the data to be allocated and the driver dataset to pilot the allocation.

Tabs on the page

Element	Description
Data to allocate	Panel for defining the measure to be allocated and the dimensional filters to be applied to the dataset to be allocated.
Driver	Panel for defining the driver measure and the dimensions of the driver with which to allocate the amount in the target.

Data to allocate

Element	Description
Operation information	Shows contextual information on the operation. • **Code**: operation ID • **Description**: operation description
Data to allocate information	Shows the contextual information on the data to be allocated. • **Dataset to be allocated**: ID of the dataset to be allocated • **Typology**: ID of the dataset typology to be allocated
Code	ID of the allocation details
Description	Description of the allocation details
Measure	Allows you to select the dataset field containing the amount to be allocated. Only one measure, one numeric field of the dataset, can be specified.
Filters on dimensions	Allows you to select the filters on the dimensions present in the dataset to be allocated (except for Scenario, Period, Entity, Var. - Det. type and Currency). For every dimension, several of the lowest level elements or nodes belonging to any hierarchy of that dimension can be selected.

Driver

Element	Description
Driver information	Shows the contextual information on the driver. • **Driver dataset**: ID of the driver dataset • **Typology**: ID of the driver dataset typology
Driver measure	Allows you to select the driver dataset field containing the amount to be used as the driver. Only one measure, a numeric field, can be specified.
Allocate on	Allows you to select the dimensions, present on the driver dataset, to which to allocate the amount (except for Scenario, Period, Entity, Var. - Det. type and Currency).
Filters	Indicates the dimensions with which to partialise the receivers' domain. If this occurs, in the data processing step, the system only considers elements that have values on the driver and form part of the defined domain.
Other filters based on data to allocate	For every managed dimension, this indicates how to partialise the specified driver based on the data to be allocated. • **None**: the system does not apply any filter to the driver (default value). • **Filter**: the system allocates the data to be processed with a driver that is filtered on the same dimension on which the data to be allocated is defined.

Special method

Function

The *special method* is an operation based on an SQL script set up by the configurator according to the project needs. In addition to the standard SQL, the special method supports CCH Tagetik system parameters, workspace parameters and reporting parameters. The use of special methods is recommended when performing operations that are more in-depth than standard CCH Tagetik operations.

System parameters supported in the operation syntax

The system parameters that are defined when the operation is performed are listed below:

Parameter	Description
System parameters	See "System parameters" on page 6.
warning_message	(optional) Output variable which returns a warning message, to be used for special method operations with "Warning" outcome (the outcome is "Successful" by default)

Workspace parameters can be referred to using the syntax *@PARAMETERCODE@* and reporting parameters can be referred to using the syntax *${@PARAMETER}*.

Special method parameters

Customized parameters can be associated with the special method, and their values are defined in DTP settings. In particular, you can set parameters to select a dataset and a type on which to run the operation, or parameters that allow you to indicate a string. Parameters can refer to the body of the special method using the syntax **&PARAMETERCODE&** (see Parameters Pane).

To define the special method operation, the following parameters must be set :

- dataset and typology, for dataset/typology parameters
- a free field, for string parameters

Execution

For every Special Method, you can specify whether to disable concurrency checks.
By default, an activity which contains at least one special method is run in isolation, i.e. disabling the activity's parallel data processing. By setting the "*Deactivate concurrency check*" flag, the user setting the parameters takes on responsibility for ensuring that the activity containing the special method can be run concurrently with other activities.

Check with formula

Function and purpose of the operation

The *Check with formula* operation allows you to use an SQL formula to run checks on the rows present in the dataset.

Operation behaviour

The following scenarios may occur during execution:

- **Rows comply with the formula**: the operation is successful.
- **Rows do not comply with the formula**: the operation returns a warning, the system displays in the audit the error message along with the details of the top 10 rows in the dataset that contain anomalies.

***IMPORTANT**: If the DTP is launched from the cockpit and anomalies are detected, the system generates a report file of the anomalies found. For further details see DTP Report.*

If the Dataset to be checked is the following:

Dataset

Category	Product	Price	Currency
Electronics	Smartphone	699	USD
Fashion	Sneakers		USD
Home	Coffee Maker	89	USD
Toys	Puzzle Game		USD

Example 1: check with formula that complies with the rows involved

Formula specified on the operation page

CURRENCY = 'USD'

The operation is successful, all rows have USD as currency and no anomaly.

Example 2: check with formula that does not comply with at least one row

Formula specified on the operation page

PRODUCT LIKE 'S%'

The operation flags rows that do not comply with the formula with anomaly.

The data processing ends with a warning highlighting in the audit the top 10 rows that do not comply with the formula:

Category	Product	Price	Currency
Home	Coffee Maker	89	USD
Toys	Puzzle Game		USD

Check with dictionary

Function and purpose of the operation

The **Check with dictionary** operation checks that the content of a dimension associated with a dictionary is consistent with the dictionary itself. For every dimension, the operation checks that its values are coherent for the entire defined dictionary.

If the option **All fields with dictionary** is selected on the operations page, the system checks all fields containing a dictionary.

***Note**: Does not handle dependent dictionaries.*

***IMPORTANT**: When the operation detects values that are not present in the dictionary for at least one dimension, the system displays an error message in the audit along with the top 10 rows containing the missing codes.*

Operation behaviour

The following scenarios may occur during execution:

- **No inconsistencies detected**: the operation is successful.
- **Inconsistencies detected**: the operation returns a warning, the system displays in the audit the error message along with the details of the top 10 rows in the dataset with inconsistencies.

***IMPORTANT**: If DTP is launched from the cockpit and inconsistencies are found, the system generates a report file of the anomalies found. For further details see DTP Report.*

Example without differences

Dataset

Name	Surname	Age	Region
Marco	Rossi	34	Lombardia
Elena	Bianchi	29	Toscana
Luca	Gallo	41	Lazio

Where the dictionary checks the Region column and contains all Italian regions.

The operation is successful and no difference is reported.

Example with differences

Dataset

Name	Surname	Age	Region
Marco	Rossi	34	Lombardia
Elena	Bianchi	29	Toscana
Luca	Gallo	41	Lazio
Sara	Neri	36	France
Chiara	Fontana	38	Italy

Where the dictionary checks the Region column and contains all Italian regions.

The data processing finds differences and ends with a warning highlighting the top 10 differences in the audit:

Name	Surname	Age	Region
Sara	Neri	36	France
Chiara	Fontana	38	Italy

Check with rule

Function and purpose of the operation

The *Check with rule* operation allows you to validate data using user-defined coherence/existence/congruence rules (see Checking rules page).

Operation behaviour

The following scenarios may occur during execution:

- **No anomaly detected**: the operation is completed successfully.
- **Anomalies detected**: the operation generates a warning. The audit shows the error message and details of the top 10 rows in the dataset with anomalies.

***IMPORTANT**: If the DTP is launched from the cockpit and anomalies are detected, the system generates a report file of the anomalies found. For further details see DTP Report.*

Check missing keys

Function and purpose of the operation

The *Check Missing Keys* operation allows you to check whether the rows in a dataset have the same values as the source dataset on the columns indicated on the Matching Fields page.

To use this operation, the user must indicate at least one field relationship, or a condition column on which to run the check.

Note: If both source and target have a **null** value for a column used as a condition, the system treats them as two different rows. Therefore, the condition is not satisfied.

Operation behaviour

The following scenarios may occur during execution:

- **No difference detected**: the operation is completed successfully.
- **Differences detected between datasets**: the operation generates a warning. The system displays an error message in the audit and the detail of the top 10 rows present in the target dataset but not in the source dataset.

***IMPORTANT**: If the DTP is launched from the cockpit and there are differences, a report file of the anomalies found is created. For further details see DTP Report.*

Example without differences

Dataset

Category	Product	Price
Electronics	Laptop	1329
Home	Air Purifier	199
Books	Novel	25

Matching Fields Table

Target field	Source
Category	Category
Product	Product

Source Dataset

Category	Product	Price
Electronics	Laptop	1129
Electronics	Bluetooth Speaker	120
Home	Air Purifier	189
Toys	Remote Control Car	50
Books	Novel	18

If the operation is successfully completed, the system shows no differences.

Example with differences

Dataset

Category	Product	Price
Electronics	Laptop	1329
Electronics	Smartphone	800
Toys	Puzzle Game	15
Home	Air Purifier	199
Books	Fantasy	20

Matching Fields Table

Target field	Source
Category	Category
Product	Product

Source Dataset

Category	Product	Price
Electronics	Laptop	1129
Electronics	Bluetooth Speaker	120
Toys	Puzzle Game	20
Toys	Remote Control Car	50
Books	Novel	18

The data processing finds differences between the tables and ends with a warning highlighting the top 10 differences in the audit:

Category	Product	Price
Electronics	Smartphone	800
Home	Air Purifier	199
Books	Fantasy	20

Existence check

Functionality and purpose of the operation

The Check - e**xistence operation che**cks for the presence of a value on the target dataset fields. This check uses the condition columns and existence columns specified on a reference (existence) dataset that has been specifically prepared (see Reference dataset).

The existence columns on the reference dataset contain an *M* (Mandatory) for the fields to be checked on the target dataset, whereas they are empty for fields that should not be considered.

The checks and conditions are applied to the target dataset using column names, which will be the same as the corresponding columns in the reference dataset.

Operation behaviour

The following scenarios may occur during execution:

- **No null value** in the dataset: the operation is completed successfully.
- **Null values detected:** the operation generates a warning. The audit shows the error message and details of the top 10 rows in the dataset with anomalies.

***IMPORTANT**: If the DTP is launched from the cockpit and there are anomalies, an anomaly report file is created. For further details see DTP Report.*

If the Dataset to be checked is the following:

Dataset

Category	Product	Price	Currency
Electronics	Smartphone	699	USD
Fashion	Sneakers		USD
Home	Coffee Maker	89	USD
Fashion	Leather Jacket	159	
Toys	Puzzle Game		USD
Books	Mystery Novel	18	

Example 1: Check with mandatory fields with values

Reference dataset - Existence

Category	Product	Price	Currency
Home	Coffee Maker	M	M
Electronics	Smartphone	M	M

The operation is successful and no anomaly is reported as all fields defined as mandatory have a value in the Dataset.

Example 2: Check with mandatory fields without any value

Reference dataset - Existence

Category	Product	Price	Currency
Home	Coffee Maker	M	M
Electronics	Smartphone	M	M
Books	Mystery Novel	M	M

The operation reports an anomaly for the row that has no value for the Currency field defined as mandatory in the existence type reference dataset.

The data processing ends with a warning, highlighting the top 10 differences in the audit:

Category	Product	Price	Currency
Books	Mystery Novel	18	

Example 3: check with optional fields

Reference dataset - Existence

Category	Product	Price	Currency
Home	Coffee Maker	M	M
Electronics	Smartphone	M	M
Books	Mystery Novel	M	

The operation does not check the Currency field for the Books - Mystery Novel condition as the field on the existence type reference dataset is empty.

The operation is successful and no anomaly is reported as all fields defined as mandatory have a value in the Dataset.

Check multiple keys

Function and purpose of the operation

The ***Check multiple keys*** operation verifies whether there are duplicate rows in the target dataset based on the user-defined keys.

In order to define the keys to be taken into account for finding duplicate rows, it is necessary to access the **Key Fields** link.

***Note**: Key fields are not mandatory. If not defined, all user fields in the target dataset will be considered as keys.*

Operation behaviour

The following scenarios may occur during execution:

- **No duplicate rows**: the operation is completed successfully.
- **Duplicate rows detected**: the system generates a warning.
  The audit shows:
  - an error message
  - the detail of the first 10 duplicate rows in the dataset
  - the number of occurrences for each row

Execution from cockpit

If the DTP is launched from the cockpit and there are duplicate rows, an anomaly report file is created. For further details see DTP Report.

In the detected anomalies, there will be as many key combinations that have generated duplicates as there are occurrences.

Example 1: no key field defined

Target Dataset

PRODUCT	PRICE	VALUE
Prod 01	3	15
Prod 01	3	15
Prod 02	5	20

The *Check Multiple* Keys *operation restores* an anomaly:

PRODUCT	PRICE	VALUE	OCCURRENCES
Prod 01	3	15	2

Example 2: a defined key field

Target Dataset

PRODUCT	PRICE	VALUE
Prod 01	3	15
Prod 01	4	21
Prod 02	5	20

On the *Key Fields* page, the PRODUCT field was specified as the key.
Duplicate rows will then be selected according to the values in the PRODUCT field.
The *Check Multiple* Keys *operation returns* an anomaly:

PRODUCT	OCCURRENCES
Prod 01	2

Loading to Analytical Workspace

Function

This operation allows you to define one or several fields of a target dataset based on another source dataset using Load on Analytical Workspace data processing (for more details, see Loading data from external datasources into Analytical Workspace in the Quick Data Loader module). To do this, the loading of data must be defined through datasource mapping (see Quick Data Loader - Datasource mapping page).

Details

To run the mapping, only datasources defined on the target dataset of the current activity can be selected.

It is also possible to select datasources with the following settings:

Source type = File

File location = Upload

In this case, mapping can only be run within a process with dimension by step (see Process with dimension by step) and not in a process by Entity (see Process by Entity).

In case of "*Execution independent of context*", if the DTP contains a Loading to Analytical Workspace operation that reads from an Analytical Workspace calculation layer, the data processing is not run and an error message is displayed.

Parameters not supported

Parameter type	Parameters
Reporting parameters in the Multi Organisation Process	• Numeric • Dimension • Simple Financial Statement Template • Advanced Financial Statement Template
System parameters	#TARGET_PARTITION#

Delete with formula

Function

This operations allows you to delete the rows present in the target dataset using an SQL formula.
It is therefore necessary to specify the SQL formula that identifies the data to be deleted.
To make the operation parametric, AW parameters can be used. (see page AW Parameters).

If the Dataset is :

Dataset

Category	Product	Price	Currency
Electronics	Smartphone	699	USD
Fashion	Sneakers		USD
Home	Coffee Maker	89	USD
Toys	Puzzle Game		USD

Example

Formula specified on the operation page

PRODUCT LIKE 'S%'

The operation deletes the rows that comply with the formula.

The data processing ends and the rows of the dataset are:

Category	Product	Price	Currency
Home	Coffee Maker	89	USD
Toys	Puzzle Game		USD

Delete multiple keys

Function and purpose of the operation

The *Delete Multiple Keys* operationallows rows in the target dataset to be deleted based on the user-defined keys.

On the operation definition page, it is possible to specify an SQL sort criterion according to which only the first row among the duplicates will be retained.

It is also possible to insert references to fields in the target dataset combined via SQL formulas and to use AW parameters (see AW Parameters page).

Note: *The criterion can be omitted. In this case, the system will use as the sorting criterion the list of all fields in the target dataset, in the order in which they were defined, and in ascending order.*

In order to define the keys to be taken into account for finding duplicate rows, it is necessary to access the **Key Fields** link.

***Note**: Key fields are not mandatory. If not defined, all user fields in the target dataset will be considered as keys.*

Example 1: no key field defined and no sort criteria specified

Target Dataset

PRODUCT	PRICE	VALUE
Prod 01	3	15
Prod 01	3	15
Prod 02	5	20

After the execution of the *Delete Multiple Keys* operation, the resulting dataset will be as follows:

Target Dataset

PRODUCT	PRICE	VALUE
Prod 01	3	15
Prod 02	5	20

Example 2: a defined key field and specified sort criterion

Target Dataset

PRODUCT	PRICE	VALUE
Prod 01	3	15
Prod 01	4	21
Prod 02	5	20

The following sorting criterion has been specified on the operations page:*PRICE desc*

On the *Key Fields* page, the PRODUCT field was specified as the key.

Duplicate rows will then be selected according to the values in the PRODUCT field and sorted according to the values in the PRICE field in descending order.

After the execution of the *Delete Multi*ple Keys *operation, the re*sulting dataset will be as follows:

Target Dataset

PRODUCT	PRICE	VALUE
Prod 01	4	21
Prod 02	5	20

Delete missing keys

Function and purpose of the operation

The *Delete Missing Keys* operation deletes rows in a dataset that do not have the same values as the source dataset on the columns indicated on the Matching Fields page.

To use this operation, the user must indicate at least one field relationship, or a condition column on which to run the check.

***Note**: If both source and target have a null value for a column used as a condition, the system treats them as two different rows. Therefore, the condition is not satisfied.*

Advanced Tab

The **source scenario** /period fields allow you to define a scenario/period from which to read data for the given source dataset. It is possible to specify a value using the AW parameters or through the same syntax used in the definition of cell fields.

When the field is left blank, the DTP launch scenario/period is taken into account; however, in the case of a non-existent or inactive scenario/period, the DTP will go ahead without running the operation, reporting the failed execution in the audit.

Note: the advanced tab is only visible for physical or virtual datasets.

Example

Target Dataset

Category	Product	Price
Electronics	Laptop	1329
Electronics	Bluetooth Speaker	99
Electronics	Smartphone	800
Toys	Puzzle Game	15
Home	Air Purifier	199
Books	Fantasy	20
Books	Novel	18

Matching Fields Table

Target field	Source
Category	Category
Product	Product

Source Dataset

Category	Product	Price
Electronics	Laptop	1129
Electronics	Bluetooth Speaker	120
Toys	Puzzle Game	20
Toys	Remote Control Car	50
Books	Novel	18

After data processing, the target dataset becomes:

Category	Product	Price
Electronics	Laptop	1329
Electronics	Bluetooth Speaker	99
Toys	Puzzle Game	15
Books	Novel	18

Insert with calculation rule

***IMPORTANT**: This operation can only be displayed and used with the licence for the ESG module.*

Function

The *insert with calculation rule* operation allows you to perform complex calculations, working on values by source dataset row, instead of by column. In particular, given a source dataset and a set of calculation rules, this operation allows you to insert the rows generated by the calculations within the target dataset.

For this type of operation, specific attributes can be specified. See Operations page for a description.

Given an operation, for every statement of a script, the system runs the defined calculation, considering the logical dimensions of every operand, both input and output. As for Custom MD Calculations, the system can combine several uniform, non-uniform or separate operands or operands without logical dimensions, grouping the information present on the data and applying the rules of the MD syntax.

Logical dimensions and scripts are defined in the calculation rule scripts page (see "Calculation rules page" on page 15).

For a list of the functions that can be run using calculation rules, see Functionalities supported by the calculation rules.

***Note**: The set of scripts to be run for the operation are determined by the system when DTP is launched.*

All scripts that have the label specified in the operation are taken into account, and among them are taken those that have the active flag enabled; scripts that have the execution condition satisfied are chosen. If start and/or end dates are specified, they must be valid in the range of the period scenario. Start and end dates are only checked when a scenario/period submission process is launched.

Example 1: output and input operands with logical dimensions on inverted columns

Source Dataset

Operand Field	Column_1	Column_2	Unit_Measurement	Value Field
OPERAND_INPUT	North	Woman	m	50
OPERAND_INPUT	North	Men	Km	40
OPERAND_INPUT	Center	Woman	m	70
OPERAND_INPUT	Center	Men		30
OPERAND_INPUT	South	Woman	Km	55
OPERAND_INPUT	South	Men	m	35
OPERAND_INPUT	East	Rather not say	dm	9
OPERAND_INPUT	East	Rather not say	dm	10
OPERAND_INPUT	East	Rather not say	dm	11

Logical dimensions table

Operand Code Script	Logical Dimension	Dataset Field
OPERAND_INPUT	GEOGRAPHIC_AREA	Column_1
OPERAND_INPUT	GENDER	Column_2
OPERAND_OUTPUT	GEOGRAPHIC_AREA	Column_2
OPERAND_OUTPUT	GENDER	Column_1

Script

```
TGKML 1.0
[TARGET].[OPERAND_OUTPUT] := [SOURCE].[OPERAND_INPUT];
```

Target Dataset

Operand Field	Column_1	Column_2	Value Field
OPERAND_OUTPUT	Woman	North	50
OPERAND_OUTPUT	Men	North	40
OPERAND_OUTPUT	Woman	Center	70
OPERAND_OUTPUT	Men	Center	30

Operand Field	Column_1	Column_2	Value Field
OPERAND_OUTPUT	Woman	South	55
OPERAND_OUTPUT	Men	South	35
OPERAND_OUTPUT	Rather not say	East	9
OPERAND_OUTPUT	Rather not say	East	10
OPERAND_OUTPUT	Rather not say	East	11

Example 2: output and input operands with non-uniform logical dimensions

Source Dataset

Operand Field	Column_1	Column_2	Unit_Measurement	Value Field
OPERAND_INPUT	North	Woman	m	50
OPERAND_INPUT	North	Men	Km	40
OPERAND_INPUT	Center	Woman	m	70
OPERAND_INPUT	Center	Men		30
OPERAND_INPUT	South	Woman	Km	55
OPERAND_INPUT	South	Men	m	35
OPERAND_INPUT	East	Rather not say	dm	9
OPERAND_INPUT	East	Rather not say	dm	10
OPERAND_INPUT	East	Rather not say	dm	11

Logical dimensions table

Operand Code Script	Logical dimension	Dataset Field
OPERAND_INPUT	GEOGRAPHIC_AREA	Column_1
OPERAND_INPUT	GENDER	Column_2
OPERAND_OUTPUT	GENDER	Column_1

Script

```
TGKML 1.0
[TARGET].[OPERAND_OUTPUT] := [SOURCE].[OPERAND_INPUT];
```

Calcolo Operand_output

The system copies the values to the output and then groups the rows with the same value by the logical dimension 'GENDER'.

Target Dataset

Operand Field	Column_1	Value Field
OPERAND_OUTPUT	Woman	175 = (50+70+55)
OPERAND_OUTPUT	Men	105 = (40+30+35)
OPERAND_OUTPUT	Rather not say	30 = (9+10+11)

Audit information

The following information is shown in the Audit window:

- within the operation, every script associated with the label inserted in the configuration step
- within every script, each statement contained in the script
- for every statement, the details of the query run in read mode and write mode

Use Reference dataset

In the operation setup step, it is possible to set a reference dataset to be used to run an implicit conversion of the values of the source dataset read in every statement of the rule script.

This dataset must have:

- a single field with **Attribute** reference type, which must contain the conversion factor to be multiplied to the original data, by its conditions.

- a single field without reference type **with the same name as the field used as the operand field** in the source and target datasets in order to implicitly associate the output operand with the corresponding conversion factor.
- one or more **Condition** reference type fields to be able to identify the correct conversion factor

As is the case when using reference datasets in other DTP operations, the condition fields are automatically related by code to the source dataset fields and "special" conditions can be used by inserting values with the '%' character in the reference dataset.

For example, A% means everything that starts with A. Thus, instead of equality between the values, the condition defined through the SQL like operator (%) will be used. A null value, however, indicates the valid relationship for any value.

***IMPORTANT**: This operation not only uses database resources but also works in memory, so in terms of performance, the number of rows of the source dataset being processed must be taken into account. In the case of conversion, the number of rows of the reference dataset could also have a major impact on performance (from the tests performed, the memory consumption of the application server is around 500 MB for a case in which the source dataset has a few million rows and produces an output of around 100,000 rows).*

For the previous examples, same source dataset if

Example 1

Source Dataset

Operand_Field	Column_1	Column_2	Unit_Measurement	Value Field
OPERAND_INPUT	North	Woman	m	50
OPERAND_INPUT	North	Men	Km	40
OPERAND_INPUT	Center	Woman	m	70
OPERAND_INPUT	Center	Men		30
OPERAND_INPUT	South	Woman	Km	55
OPERAND_INPUT	South	Men	m	35
OPERAND_INPUT	East	Rather not say	dm	9
OPERAND_INPUT	East	Rather not say	dm	10
OPERAND_INPUT	East	Rather not say	dm	11

Script

```
TGKML 1.0
[TARGET].[OPERAND_OUTPUT] := [SOURCE].[OPERAND_INPUT];
```

Reference dataset

Operand_Field	Unit_Measurement	Conversion_Factor	Filter_Column	Filter_Column
OPERAND_OUTPUT	m	10		0
OPERAND_OUTPUT	Km	0.1		0
OPERAND_OUTPUT	cm	1000		0
OPERAND_OUTPUT	dm	10		0
OPERAND_OUTPUT	dm	100	Value_Field > 9	1
OPERAND_OUTPUT	dm	1000	Value_Field > 10	2

Before running each statement, the system enriches the input data by filtering on the rows of the reference dataset by OPERAND_OUTPUT and multiplying the source values by the identified conversion factor.

Enriched source dataset

Operand_Field	Column_1	Column_2	Unit_Measurement	Value_Field
OPERAND_INPUT	North	Woman	m	500 = (50*10)
OPERAND_INPUT	North	Men	km	4 = (40*0.1)
OPERAND_INPUT	Center	Woman	m	700 = (70*10)
OPERAND_INPUT	Center	Men		30
OPERAND_INPUT	South	Woman	km	5.5 = (55*0.1)
OPERAND_INPUT	South	Men	m	350 = (35*10)
OPERAND_INPUT	East	Rather not say	dm	90 = (9*10)
OPERAND_INPUT	East	Rather not say	dm	1000 = (10*100)
OPERAND_INPUT	East	Rather not say	dm	11000 = (11*1000)

Calcolo Operand_output

The system copies the values to the output, keeping the splitting of logical dimensions.

Target Dataset

Operand_Field	Column_1	Column_2	Value_Field
OPERAND_OUTPUT	Woman	North	500
OPERAND_OUTPUT	Men	North	4
OPERAND_OUTPUT	Woman	Center	700
OPERAND_OUTPUT	Men	Center	30
OPERAND_OUTPUT	Woman	South	5.5
OPERAND_OUTPUT	Men	South	350
OPERAND_OUTPUT	Rather not say	East	90
OPERAND_OUTPUT	Rather not say	East	1000
OPERAND_OUTPUT	Rather not say	East	11000

Example 2

Source Dataset

Operand_Field	Column_1	Column_2	Unit_Measurement	Value Field
OPERAND_INPUT	North	Woman	m	50
OPERAND_INPUT	North	Men	Km	40
OPERAND_INPUT	Center	Woman	m	70
OPERAND_INPUT	Center	Men		30
OPERAND_INPUT	South	Woman	Km	55
OPERAND_INPUT	South	Men	m	35
OPERAND_INPUT	East	Rather not say	dm	9
OPERAND_INPUT	East	Rather not say	dm	10
OPERAND_INPUT	East	Rather not say	dm	11

Script

```
TGKML 1.0
[TARGET].[OPERAND_OUTPUT] := [SOURCE].[OPERAND_INPUT];
```

Reference dataset

Operand_Field	Unit_ Measurement	Conversion_ Factor	Filter_ Column	Filter_ Column
OPERAND_ OUTPUT	m	10		0
OPERAND_ OUTPUT	Km	0.1		0
OPERAND_ OUTPUT	cm	1000		0
OPERAND_ OUTPUT	dm	10		0
OPERAND_ OUTPUT	dm	100	Value_ Field > 9	1
OPERAND_ OUTPUT	dm	1000	Value_ Field > 10	2

Before running each statement, the system enriches the input data by filtering on the rows of the reference dataset by OPERAND_OUTPUT and multiplying the source values by the identified conversion factor.

Enriched source dataset

Operand_Field	Column_1	Column_2	Unit_Measurement	Value Field
OPERAND_INPUT	North	Woman	m	500 = (50*10)
OPERAND_INPUT	North	Men	km	4 = (40*0.1)
OPERAND_INPUT	Center	Woman	m	700 = (70*10)
OPERAND_INPUT	Center	Men		30
OPERAND_INPUT	South	Woman	km	5.5 = (55*0.1)
OPERAND_INPUT	South	Men	m	350 = (35*10)
OPERAND_INPUT	East	Rather not say	dm	90 = (9*10)
OPERAND_INPUT	East	Rather not say	dm	1000 = (10*100)
OPERAND_INPUT	East	Rather not say	dm	1000 = (10*100)

Calcolo Operand_output

The system copies the values on the output and then groups the rows with the same value for the "GENDER" logical dimension.

Target Dataset

Operand_Field	Column_1	Value Field
OPERAND_OUTPUT	Woman	125.5 = (500+700+5.5)
OPERAND_OUTPUT	Men	384 = (4+30+350)
OPERAND_OUTPUT	Rather not say	12090 = (90+1000+11000)

XML Generator

Contents

This section covers the following topics:

XML Generator: Basic Concepts 77
Regulatory reporting page 77
Tag configuration page 78

XML Generator: Basic Concepts

Introduction

The XML Generator allows you to extract data in a customized XML format. To configure data extraction, you need to complete the following steps:

- Configure the tags of the XML document.
- Map the XML tags to the fields of the dataset from which the data will be extracted.
- Configure the task to start the XML document extraction.

Details on XML Tags

The structure of XML tags can contain multiple levels of nodes and leaf tags. To define the hierarchy, you must specify a parent tag for each tag. If the parent tag reference is empty, the tag will be considered the root of the XML document.

The root is indicated by the default value 0000.

When a parent tag is deleted, its reference is automatically removed from all child tags.

If the system detects multiple root tags, you need to redefine the hierarchy to ensure there is only one root tag per document.

Mapping XML Tags to Dataset Fields

To insert dataset values into the XML file, you must map the dataset fields to the XML tags using Data Target Mapping XML. See the XML Mapping page for details.

Processing Task

To run the XML file generation process, you need to set up a workflow task that generates the XML file from the dataset in the Analytical Workspace.

See Exporting Analytical Workspace Data via Task for more information.

Regulatory reporting page

 Application database > Regulatory reporting tile

 Application database > Navbar > Reporting > Regulatory reporting

Purpose of the page

This page allows you to create a txt file containing the Bankit reporting required by the Bank of Italy, or a XML file template.

The Bankit reporting for the Bank of Italy's supervisory activities is generated on the basis of a matrix containing a list of the sub-items, or the accounts present in CCH Tagetik as they must be reported to the Bank of Italy, on its rows, and the required variables on its columns.

Page fields

Element	Description
Code	Bankit reporting code
Description	Bankit reporting description
Format type	Bankit reporting format • **Bank of Italy**(txt file) • **XML**
Tag configuration	(With **XML** format) Opens a page to configure the XML tags (see Tag Configuration page).
Application code	(With **Bank of Italy** format) ID code of the regulatory basis on which to report.
Base Type	Bankit reporting type
Classifiers	(With **Bank of Italy** format) One row is inserted in the file for every variable, and the rows are of fixed length. In addition to the value taken on by the variable, the system adds as many zeros to the row as necessary to reach the specified maximum length. **Code**: code according to the regulation that identifies the variable. **Length**: maximum number of characters for its value, according to the regulations.

Tag configuration page

Application database > Regulatory reporting tile > Tag Configuration

Application database > Navbar > Reporting > Regulatory reporting > Tag Configuration

Purpose of the page

This page allows you to create XML tags.

Page fields

Element	Description
Code	Unique ID of the tag in the XML tree (based on the parent). ***Note**: if a tag with the same name but a different parent is present in the tree, it must have a different code.*
Description	Tag description
Tag name	Tag name to be shown in the XML
Parent item	Allows you to select a tag that has already been defined to make it the parent of the current tag, creating a hierarchy. When this is not defined, the current tag is considered the XML document's root tag. The XML document can only have one root tag.